Machine Learning Project: Speech Emotion Recognition System

Executive Summary

This report presents a comprehensive study on **Speech Emotion Recognition (SER)**, developing a system that classifies **six emotions (angry, happy, sad, fear, disgust, neutral)** from audio recordings. Using the CREMA-D dataset of 7,442 audio clips, we systematically explored 15 machine learning techniques achieving a **final accuracy of 82.22%**, an improvement of 21.33% over our best baseline model (Bagging Classifier, 60.9%).

The Challenge: Recognizing emotion from speech is difficult because different emotions can sound acoustically similar. For example, both angry and happy speech tend to be loud and fast (high arousal), while both fearful and sad speech tend to be quiet and hesitant (low arousal). Our task was to find acoustic features and classification strategies that could distinguish these overlapping categories while **strictly using only the models covered in class**.

Our Approach: The project proceeded in two major phases. In Part A, we established a baseline using 162 standard acoustic features (MFCCs, mel spectrograms, chroma features, zero-crossing rate and RMS energy) with nine classical machine learning algorithms: **Logistic Regression (LR), Support Vector Machine (SVM), K-Nearest Neighbours (KNN), Naive Bayes (NB), Gaussian Mixture Model (GMM), Decision Tree (DT), Bagging, Random Forest (RF), AdaBoost.** Bagging achieved the best baseline accuracy of 60.9%. In Part B, we systematically improved upon this baseline through **enhanced feature engineering** (adding 48 features), **hyperparameter tuning** and **ensemble methods** (stacking classifiers with optimized feature selection).

Key Findings: Our best-performing approach is the **Stacking Ensemble** (`stacking_diverse`): a single-layer `StackingClassifier` whose base level combines seven tuned models (Gradient Boosting (GB), RF, KNN, Extra Trees (ET), SVM, LR and NB), with a Logistic Regression meta-learner on top. This diverse stack achieved **82.22%** accuracy. Interestingly, a more complex "Ultra Ensemble" that averaged this stack with two additional pipelines (`stacking_selected`, `GradientBoosting`) consistently under-performed the single stacking model, showing that adding a second layer of ensembling can dilute the strong, well-calibrated predictions of an already optimized stack. We also found that Naive Bayes performs very poorly on its own (23.7%, worse than random guessing). This behaviour is consistent with its strong independence assumptions being a poor match for our highly correlated MFCC/mel feature space, but despite its weak standalone performance it **still contributes useful diversity** when used as one of many base learners inside the stacking ensemble.

Report Structure: Section 1 introduces the problem and objectives. Section 2 describes the CREMA-D dataset. Section 3 (Part A) covers our baseline system including feature extraction and nine baseline models. Section 4 (Part B) details our systematic improvements. Section 5 discuss the development of a production ready machine learning application and a complete reproducibility guide. Section 6 outlines future deep learning directions and Section 7 contains references.

Table of Contents

1. BACKGROUND AND INTRODUCTION 3
1.1 PROBLEM STATEMENT AND MOTIVATION 3
1.2 RELATED WORKS 3
1.3 PROBLEM FORMULATION AND SOLUTION OVERVIEW 3
2. DATASET 4
2.1 CREMA-D OVERVIEW 4
2.2 EMOTION DISTRIBUTION 4
2.3 DATA EXPLORATION 5
3. PART A: BASELINE SYSTEM – APPLYING CLASSROOM KNOWLEDGE 7
3.1 DATA AUGMENTATION 7
3.2 BASELINE FEATURE EXTRACTION (162 FEATURES) 7
3.3 BASELINE MODELS (WITHOUT HYPERPARAMETER TUNING) 8
3.4 BASELINE MODELS (WITH HYPERPARAMETER TUNING) 10
3.5 PART A SUMMARY AND CONCLUSION 12
4. PART B: SYSTEMATIC IMPROVEMENTS 14
4.1 KEY SOURCES OF PERFORMANCE IMPROVEMENTS 15
4.2 RETRAINING BASELINE MODELS WITH ALL IMPROVEMENTS 18
4.3 FINDING THE OPTIMAL ENSEMBLE – `STACKING_DIVERSE` 19
4.4 CAN WE DO BETTER WITH AN ULTRA ENSEMBLE (ENSEMBLE OF ENSEMBLES)? 20
4.5 OVERALL CONCLUSION 21
5. DEVELOPING A PRODUCTION-READY MACHINE LEARNING APPLICATION 21
5.1 APPLICATION OVERVIEW 22
5.2 SOURCE CODE / STEPS TO REPRODUCE MODELS AND PLOTS 22
5.3 APPLICATION SETUP & DEPLOYMENT GUIDE 23
5.4 APPLICATION ARCHITECTURE 26
5.5 SOFTWARE DEVELOPMENT LIFECYCLE (SDLC) 27
5.6 DEVELOPMENT METHODOLOGY 28
5.7 CODE QUALITY STANDARDS 28
5.8 MODEL VERSIONING 28
5.9 TESTING 29
5.10 INFRASTRUCTURE AS CODE (IAC) 29
5.11 MODEL DEVELOPMENT LIFECYCLE 30
5.12 CICD 31
5.13 MODEL PERFORMANCE MONITORING 33
5.14 OBSERVABILITY 36
6. FUTURE WORK: DEEP LEARNING APPROACHES 37
7. REFERENCES 38

1. Background and Introduction

1.1 Problem Statement and Motivation

Human communication conveys far more than words alone. Research in psychology suggests that up to 38% of emotional meaning in face-to-face communication comes from vocal tone rather than the words themselves (Mehrabian, 1981). When we hear someone say "I'm fine", we instinctively recognize whether they are genuinely content, masking sadness or expressing irritation – not from the words, but from how they are spoken. This ability to perceive emotion from voice is fundamental to human social interaction, yet it remains challenging to automate.

How It Would Help Society:

The ability to automatically recognize emotions from speech has transformative potential across multiple domains. In mental health, voice-based emotion detection could enable continuous, non-invasive monitoring of patients with depression or anxiety, providing early warning signs of deteriorating conditions without requiring active patient engagement.

In customer service, emotion-aware systems could automatically escalate calls where the customer sounds frustrated, route them to more experienced agents or provide real-time guidance to representatives.

For assistive technologies, emotion recognition could help individuals with autism spectrum disorder navigate social situations or enable more empathetic interactions between elderly users and care robots.

This project develops an automated Speech Emotion Recognition (SER) system that classifies emotions from audio speech signals. The system extracts acoustic features from raw audio and applies machine learning algorithms to recognize six emotion categories: anger, happiness, sadness, fear, disgust, and neutral states.

1.2 Related Works

We drew inspiration from Shoshika's Kaggle notebook on speech emotion recognition which formed part of our feature extraction and classical ML pipeline design: https://www.kaggle.com/code/shoshika/speech-emotion-recognition/notebook (Shoshika, 2025). His approach achieves roughly 61% accuracy on CREMA-D. Our project improves on it by systematically exploring what can be achieved with **traditional machine learning techniques taught in our course**, documenting not just what works but why approaches succeed or fail. This yields practical insights about feature engineering and ensemble methods that remain relevant even as deep learning dominates benchmarks today.

Our best result (82.22% accuracy) comes from a diverse **Stacking Ensemble** (`stacking_diverse`) that combines seven strong classical models within a single `StackingClassifier`. It is competitive with traditional ML benchmarks while using a transparent, interpretable approach that does not require GPU resources or extensive hyperparameter tuning. A more elaborate "Ultra Ensemble" that averaged this stack with additional models was empirically found to be slightly worse, reinforcing that a single, well-designed stacking ensemble can be more effective than deeper, multi-layer ensembles.

1.3 Problem Formulation and Solution Overview

Problem Definition: Given an audio signal $x(t)$ of duration T seconds, sampled at rate f_s, the task is to predict the emotion label $y \in angry, happy, sad, fear, disgust, neutral$. Mathematically, we seek a function f: X → Y that minimizes classification error, where X is the space of audio features and Y is the set of emotion labels.

Our Solution: Our approach transforms the raw audio signal through a feature extraction pipeline that captures emotionally relevant acoustic properties while reducing dimensionality from 55,125 raw samples to 210 features. We then apply classification algorithms to map these features to emotion labels.

The solution proceeds in three stages: (1) In the feature extraction stage, we compute five categories of features (Refer to Section 3.2 for detailed description of each feature category): time-domain features (zero-crossing rate, RMS energy), frequency-domain features (mel spectrogram, chroma), cepstral features (MFCCs) and temporal dynamics features (delta and delta-delta MFCCs). (2) In the classification stage, we train and evaluate multiple algorithms including SVM, Random Forest, Bagging and ensemble methods. (3) In the optimization stage, we apply feature selection and stacking to combine classifier strengths.

We restrict our approach to traditional machine learning methods taught in class, reserving deep learning architectures for discussion as future work.

Our approach follows a **two-phase methodology**:

PHASE	DESCRIPTION	FEATURES	BEST MODEL	ACCURACY
PART A: BASELINE	Standard ML with original features, basic hyperparameter tuning	162	Bagging Classifier	60.9%
PART B: INNOVATION	Enhanced features, Gradient Boosting, hyperparameter tuning	210	`stacking_diverse` (Stacking Ensemble)	82.22%

The final system in this report adopts `stacking_diverse` as the primary model, achieving 82.22% accuracy using a single-layer stacking ensemble over seven diverse, tuned base learners. We also experimented with an "Ultra Ensemble" that averaged probabilities from two stacking models (one with feature selection) plus Gradient Boosting; however, this more complex architecture topped out around 80-81% accuracy and did not outperform the simpler stacking ensemble.

2. Dataset

2.1 CREMA-D Overview

We use the Crowd-Sourced Emotional Multimodal Actors Dataset (CREMA-D), containing 7,442 audio clips from 91 actors (48 male, 43 female) spanning various ethnicities and ages (20-74 years).

Filename Convention: {ActorID}_{Sentence}_{Emotion}_{Intensity}.wav

Example: 1001_DFA_ANG_XX.wav indicates Actor 1001, sentence DFA, Angry emotion.

2.2 Emotion Distribution

Emotion	Code	Raw Count	Percentage	After 3x Augmentation*
Angry	ANG	1,271	17.1%	3,813
Disgust	DIS	1,271	17.1%	3,813

Fear	FEA	1,271	17.1%	3,813
Happy	HAP	1,271	17.1%	3,813
Sad	SAD	1,271	17.1%	3,813
Neutral**	NEU	1,087	14.6%	3,261
Total		7,442	100%	22,326

*** 3x Augmentation:** We apply augmentation to grow the dataset and teach models invariances to real-world recording conditions. Each original clip is kept, plus two augmented variants: (i) random noise to simulate mic/background variability and misalignment, (ii) time stretch + pitch shift to simulate speaker rate/pitch variation.
**** Neutral Emotion:** Neutral class is slightly underrepresented, so we have addressed it through class weighting.

2.3 Data Exploration

Before modeling, we conducted exploratory data analysis to understand the acoustic characteristics of different emotions.

Insight 1: Waveform Amplitude Patterns

Visual inspection of waveforms revealed distinct amplitude patterns across emotions. Angry speech consistently showed high-amplitude, sustained energy throughout the utterance. Sad speech showed low amplitude with a declining envelope. Fear showed irregular, unpredictable amplitude patterns. Happy speech showed variable amplitude with more dynamic range than neutral. These observations motivated our inclusion of RMS energy features and energy slope in our feature set.

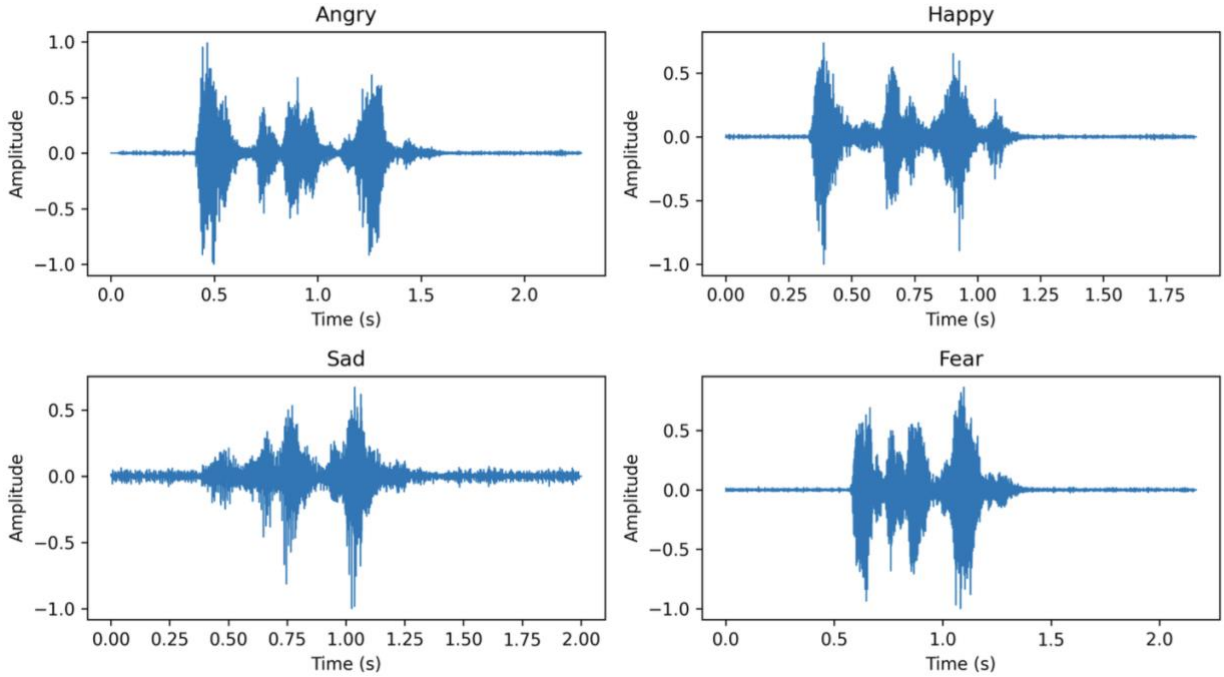


Figure 1 - Waveform Amplitude Patterns by Emotion

Insight 2: Spectral Distribution Differences

Examining spectrograms revealed that emotions differ in their frequency distribution. Angry speech showed more energy in higher frequencies (above 3kHz), giving it a "harsh" quality. Sad speech concentrated energy in lower frequencies (below 1kHz), producing a "muffled" sound. These patterns justified our use of mel spectrogram features, which capture energy distribution across frequency bands.

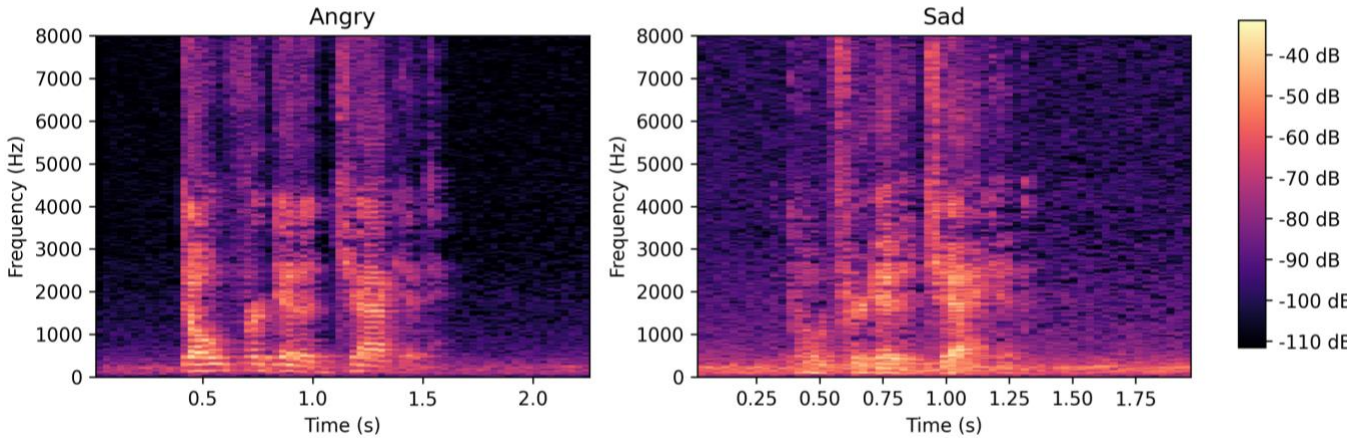


Figure 2 - Spectrograms by Emotion

Insight 3: Feature Correlation Analysis

Computing correlation matrices revealed that many features are highly correlated. Adjacent MFCC coefficients showed correlations of 0.6-0.8. Adjacent mel spectrogram bands showed correlations exceeding 0.9. This strong correlation structure is likely one factor behind the poor standalone performance of Naive Bayes, which assumes conditional independence of features. In contrast, our other models (e.g. tree ensembles, SVMs, logistic regression) do not rely on this independence assumption and can exploit correlated inputs, which is consistent with their higher accuracies.

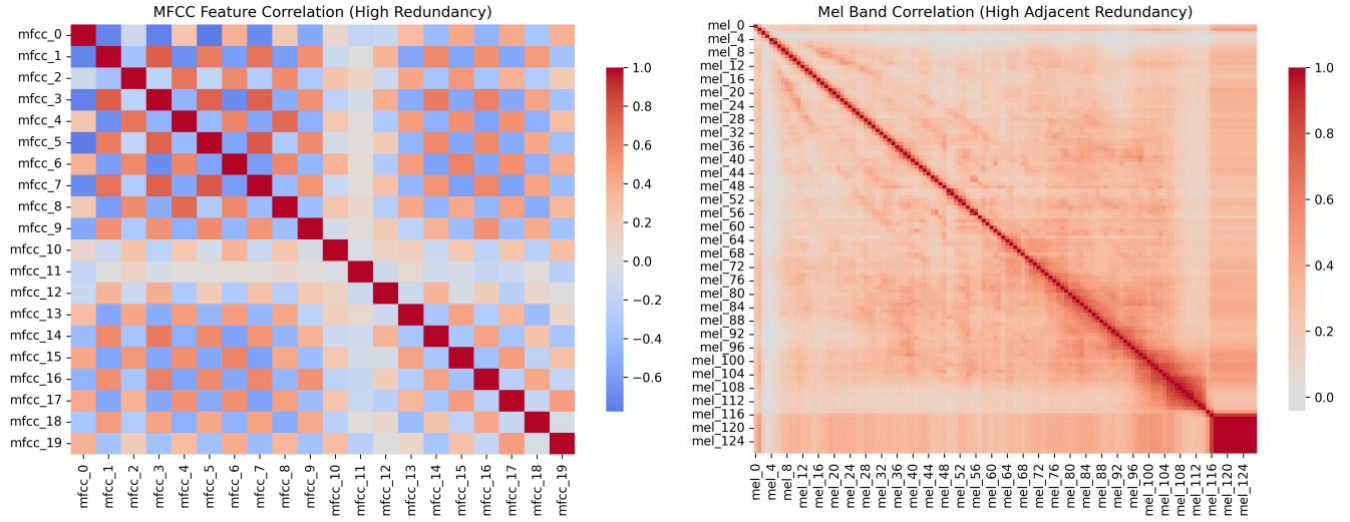


Figure 3 - MFCC Feature Correlation & Mel Band Correlation

Insight 4: Confusion Pattern Preview

Plotting feature distributions by emotion revealed substantial overlap between certain pairs. Fear and sad samples had nearly identical distributions for mean pitch and RMS energy. Angry and happy samples overlapped significantly on energy and speech rate measures.

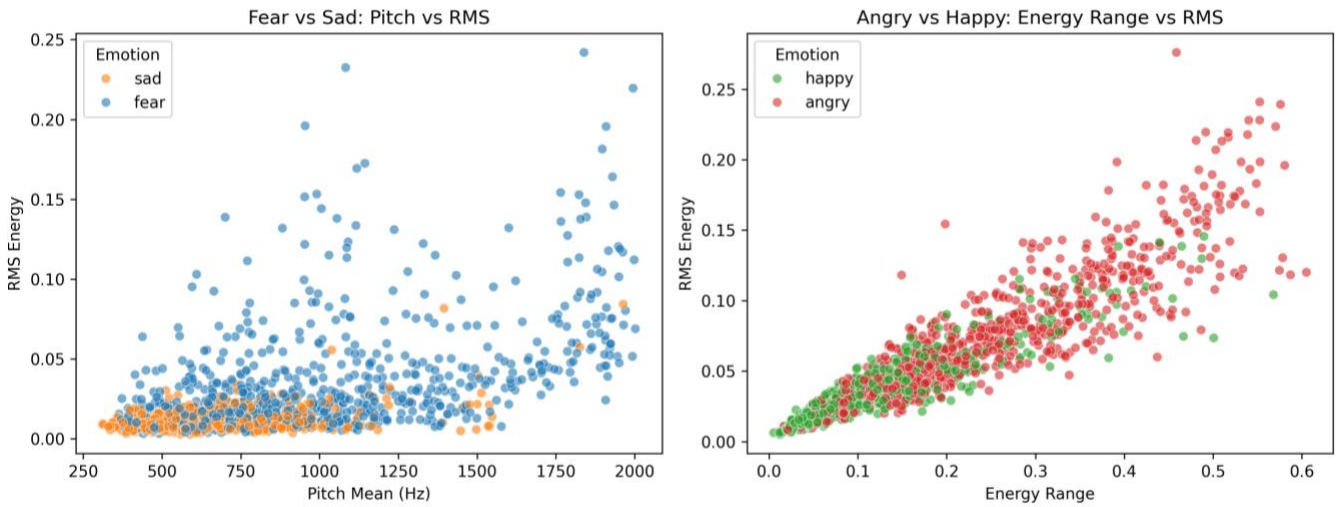


Figure 4 - Overlapping Feature Spaces

3. PART A: Baseline System – Applying Classroom Knowledge

This section documents our first approach to speech emotion recognition, applying machine learning techniques learned in class. Our goal was to establish a baseline using standard feature extraction methods and classical algorithms such as **Support Vector Machines, Bagging, Random Forest, K-Nearest Neighbours, Logistic Regression, Naive Bayes etc**. By evaluating these models, we aimed to understand which techniques transfer well to the audio domain and to identify specific challenges/failure that would guide our subsequent experimentation in Part B.

3.1 Data Augmentation

To increase training data diversity and improve model robustness to real-world variations, we apply augmentation to grow the dataset. Data augmentation is an increasingly popular way to increase the limited amount of good quality audio training data and can enhance emotion classification performance significantly (Avci, 2025). Each original clip is kept, plus two augmented variants, giving a 3x expansion:

- **Variant 1 (Random Noise):** Add random noise to simulate mic/background variability and misalignment.
- **Variant 2 (Time Stretch + Pitch Shift):** Time stretch + shift pitch slightly to simulate more realistic speaker pitch variation.

3.2 Baseline Feature Extraction (162 Features)

We extract a comprehensive set of acoustic features that capture different aspects of speech. This section explains what each feature represents and why it matters for emotion recognition.

1. **Zero Crossing Rate (ZCR) – 1 feature:** The rate at which the audio signal changes from positive to negative (or vice versa).

   a. **High ZCR:** Indicates noisy, unvoiced sounds (like "s", "f", "sh") or rough/breathy
   b. **Anger:** Often has higher ZCR due to harsh, tense vocal quality
   c. **Sadness:** Lower ZCR due to smoother, more voiced sounds
   d. **Fear:** Variable ZCR due to irregular breathing patterns

2. **Root Mean Square Energy (RMS) – 1 feature:** A measure of the signal's loudness/energy, computed as the square root of the mean of squared amplitude values.
   a. **High RMS:** Anger, happiness, excitement (high arousal emotions)
   b. **Low RMS:** Sadness, fear, neutral (low arousal or subdued emotions)
   c. RMS directly correlates with perceived loudness and vocal effort

3. **Mel Spectrogram – 128 features:** A representation of how energy is distributed across different frequencies over time, scaled to match human hearing perception.
   a. Different emotions have distinct spectral "fingerprints"
   b. **Anger:** More energy in higher frequencies (harsh, bright sound)
   c. **Sadness:** Energy concentrated in lower frequencies (dark, muffled sound)
   d. **Fear:** Irregular spectral patterns, trembling affects multiple bands
   e. The 128 mel bands capture fine-grained frequency information

4. **Chroma Features – 12 features:** Energy distribution across the 12 pitch classes of Western music (C, C#, D, D#, E, F, F#, G, G#, A, A#, B) regardless of octave.
   a. Captures the harmonic/tonal content of speech; different emotions produce different pitch patterns
   b. **Happy:** More varied chroma, melodic speech
   c. **Sad:** Limited chroma variation, monotonous
   d. **Angry:** Sharp chroma changes, emphatic pitch movements

5. **Mel-Frequency Cepstral Coefficients (MFCCs) – 20 features:** The most important features in speech processing. MFCCs capture the shape of the vocal tract (how the mouth, tongue and throat are positioned), which determines the "color" or timbre of the voice.
   a. Different emotions produce distinct vocal tract configurations
   b. **Anger:** Tense vocal cords → specific MFCC pattern
   c. **Sadness:** Relaxed, breathy voice → different pattern
   d. **Fear:** Constricted throat → yet another pattern
   e. MFCCs are speaker-normalized due to the cepstral analysis

3.3 Baseline Models (without Hyperparameter Tuning)

We implemented nine classical machine learning algorithms. Each takes the 162-dimensional feature vector as input and outputs one of the 6 emotion labels.

The baseline results show **Bagging Classifier as the winner** among traditional ML models, with **Naive Bayes performing the poorest.**

Model	Accuracy	Training Time
Bagging Classifier	**58.78%**	**379.52s**
Random Forest (RF)	56.52%	57.54s
Support Vector Machine with RBF Kernel (SVM RBF)	47.58%	54.84s
Decision Tree	45.72%	12.50s
SVM (Linear)	45.03%	90.15s
Logistic Regression	43.89%	0.63s
Adaboost	39.09%	115.40s
K-Nearest Neighbours (KNN)	38.40%	0.47s
SVM (Polynomial)	37.03%	54.41s
Naive Bayes (Gaussian)	23.00%	0.07s

Why Bagging and Random Forest Were the Best Baselines

Bagging performed the best at 58.78% with Random Forest (RF) close behind. Both models benefit from the same properties:

1. **Both excel at capturing complex feature interactions**. Emotions do not manifest through single acoustic features in isolation; rather, they emerge from complex combinations of multiple features. For example, angry speech is characterized by the simultaneous presence of high RMS energy, wide pitch range, and elevated values in higher-frequency mel bands. A single decision tree can capture this through nested conditions such as "if RMS > 0.5 AND mel_band_80 > 0.3 AND pitch_range > 50 Hz, then predict angry." By training 100 such trees, each exploring different feature combinations, the model effectively learns these multi-dimensional patterns.

2. **They are robust to noisy data and augmentation.** The ensemble averaging in both models provides robustness against the noise introduced by our data augmentation process. When we added random noise or pitch-shifted the audio samples, some augmented samples became less representative of their original emotion. A single decision tree might overfit to these noisy samples, but when 100 trees vote, the noise gets averaged out.

3. **They are insensitive to feature scaling.** Our 162 features span very different numeric ranges (e.g., small MFCC coefficients vs. large mel energies). Algorithms such as SVM and KNN are sensitive to scale and require careful preprocessing to avoid over-emphasizing certain features. Tree-based models split on relative thresholds within each feature, so they work well even when features are heterogeneously scaled. With bagging we use **all features at each split**, so each tree can exploit the full feature set. In contrast, RF forces trees to consider only a random subset of features per split. This usually helps reduce correlation between trees, but sometimes it may prevent the model from seeing the MFCC/mel/RMS combinations, which likely explains the slight performance gap in favour of Bagging.

Why Naive Bayes Performed the Worst

Naive Bayes achieved only 23.00% accuracy. This poor performance suggests a mismatch between the algorithm's assumptions and our feature space. The core assumption of Naive Bayes is that all features are conditionally independent given the class label. However, our audio features are highly correlated, violating this assumption in three major ways.

First, we found out that MFCC coefficients move together. Adjacent MFCCs often rise and fall as a group because they capture similar parts of the sound spectrum. So instead of giving the model several different clues, they mostly tell it the same thing.

Second, the mel spectrogram bands are also strongly correlated (see Section 2.3 Insight 3). Nearby bands overlap by design, so when there's strong energy at a certain frequency, multiple bands spike together. Naive Bayes treats these as many independent signals even though they reflect the same event in the audio.

Third, some feature types measure related properties. For example, RMS energy and the overall energy in the mel spectrogram are almost the same information expressed in two different ways.

Because of these correlations, Naive Bayes effectively 'double-counts' evidence, often producing over-confident but incorrect posteriors. Adjusting `var_smoothing` did not materially improve performance, so Naive Bayes remains the weakest model in our setting.

3.4 Baseline Models (with Hyperparameter Tuning)

We performed a focused round of hyperparameter tuning. Rather than exhaustively explore huge search spaces, we used small, interpretable grids informed by standard defaults and domain intuition, and we evaluated them with stratified 3-fold cross-validation. A more comprehensive hyperparameter tuning will be carried out in Part B of the experiment.

Hyperparameter Tuning Setup

All tuning was done in `PART A – Baseline Models.ipynb` using `GridSearchCV` with `StratifiedKFold (n_splits=3, shuffle=True, random_state=42)`. For each model, we defined a compact grid:

- **Logistic Regression**
  - Grid: C ∈ {0.1, 1.0, 10.0}, solver='lbfgs', max_iter=1000
  - Best: C = 0.1, solver='lbfgs', max_iter=1000
- **SVM (Linear)**
  - Grid: C ∈ {0.1, 1.0, 10.0}
  - Best: C = 1.0
- **SVM (RBF)**
  - Grid: C ∈ {1, 10, 50}, gamma ∈ {'scale', 'auto'}
  - Best: C = 50, gamma = 'auto'
- **SVM (Polynomial)**
  - Grid: C ∈ {0.1, 1.0, 10.0}, degree ∈ {2, 3}, gamma ∈ {'scale', 'auto'}
  - Best: C = 10.0, degree = 2, gamma = 'auto'
- **Decision Tree**
  - Grid: max_depth ∈ {10, 20, 30, None}, min_samples_split ∈ {2, 5, 10}

  - Best: max_depth = None, min_samples_split = 2
- **Random Forest (Bagging)**
  - Grid: n_estimators ∈ {100, 200}, max_depth ∈ {20, 30, None}, max_features ∈ {'sqrt', 'log2'}
  - Best: n_estimators = 200, max_depth = None, max_features = 'sqrt'
- **Bagging Classifier**
  - Grid: n_estimators ∈ {50, 100, 150}, max_samples ∈ {0.5, 0.8, 1.0}
  - Best: n_estimators = 150, max_samples = 1.0
- **K-Nearest Neighbours (KNN)**
  - Grid: n_neighbors ∈ {5, 7, 9}, weights ∈ {'uniform', 'distance'}
  - Best: n_neighbors = 9, weights = 'distance'
- **Naive Bayes (Gaussian)**
  - Grid: var_smoothing ∈ {1e-9, 1e-8, 1e-7}
  - Best: var_smoothing = 1e-9

For each grid, we selected the configuration with the highest cross-validated accuracy, retrained the best estimator on the full training set and evaluated it on the same held-out test set as the baseline models.

The table below summarizes **original vs. tuned test accuracy** for the nine tuned classifiers:

Model	Original Test Accuracy	Tuned Test Accuracy	Accuracy	Best Hyperparameter
Bagging Classifier	58.7%	**60.9%**	**+2.2%**	n_estimators=150, max_samples=1.0
Random Forest (Bagging)	56.5%	57.7%	+1.2%	n_estimators=200, max_depth=None, max_features='sqrt'
SVM (RBF)	47.4%	56.2%	**+8.8%**	C=50, gamma='auto'
SVM (Polynomial)	36.5%	48.0%	**+11.5%**	C=10, degree=2, gamma='auto'
SVM (Linear)	45.4%	45.4%	~0.0%	C=1.0
Logistic Regression	43.8%	43.9%	+0.1%	C=0.1
Decision Tree	43.6%	43.6%	~0.0%	max_depth=None, min_samples_split=2
K-Nearest Neighbours	38.1%	40.7%	+2.7%	n_neighbors=9, weights='distance'
Naive Bayes (Gaussian)	23.1%	23.1%	~0.0%	var_smoothing=1e-9

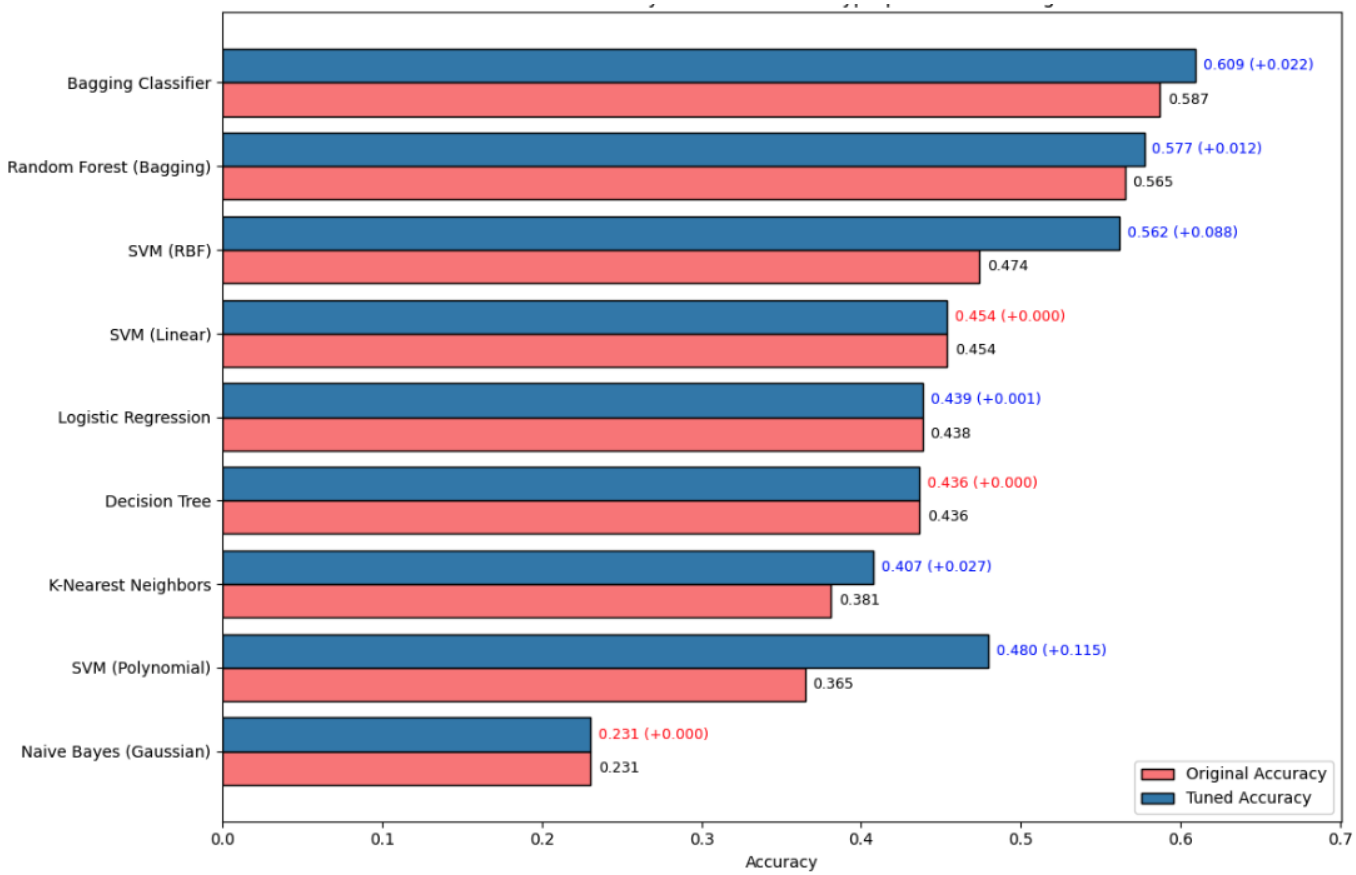


Figure 5 - Model Accuracy Before and After Hyperparameter Tuning

Large Gains: SVM (Polynomial) and SVM (RBF)

The largest improvements came from the non-linear SVMs (Polynomial and RBF). By allowing degree 2 polynomials with higher C and gamma='auto' for example, unlocked SVM (Polynomial)'s ability to model complex interactions between audio features. These jumps confirm the **default hyperparameters can severely under-represent** what SVMs are capable of.

Moderate Gains: Bagging, Random Forest, KNN

We see Bagging, Random Forest and KNN having moderate improvements. Taking Bagging as an example, increasing the number of estimators to 150 trees and using all samples (max_samples=1.0) reduced variance while keeping a diverse set of trees. Each tree still sees a different bootstrap sample, but having more trees means that noisy augmentation artefacts are averaged out more reliably. This tuned Bagging model becomes **our strongest single baseline in Part A**.

KNN is highly sensitive to the choice of k and the weighting scheme. Using more neighbours (k=9) with weights='distance' helps the classifier smooth out noise. Despite the gain however, KNN remains clearly inferior to the ensemble methods for complex audio data.

3.5 Part A Summary and Conclusion

Putting everything together, Part A establishes a clear and rigorous baseline for the rest of the project:

1. **Standard features + classical models reach 60.9% accuracy.**
   With 3x data augmentation and 162 standard features, our best tuned baseline (Bagging Classifier with 150 trees) achieves 60.9% test accuracy.

2. **Tree-based bagging ensembles are the most reliable classical approach.**
   Both Bagging and Random Forest consistently outperform SVMs, KNN and Naive Bayes, even after tuning. This suggest that the models naturally handle acoustic scaled features, capture complex acoustic interactions, and are robust to augmentation noise.

3. **SVMs are much better than their defaults suggest, but still not enough.**
   Properly tuning C, gamma and polynomial degree significantly boosts SVM performance, especially for the polynomial and RBF kernels. However, they remain slightly behind the best tree ensembles and are more computationally expensive, especially when extended to more features in Part B.

4. **Some model families are fundamentally mismatched to this task.**
   Naive Bayes fails due to violated independence assumptions; linear models hit a capacity ceiling even with their best C.

5. **Error patterns reveal deeper challenges.**
   Bagging Classifier correctly classified six out of ten test samples on average, but still confused four out of ten. Analysis of the confusion matrix showed that **errors concentrated in specific pairs**: fear misclassified as sad, angry misclassified as happy, and disgust misclassified as sad.

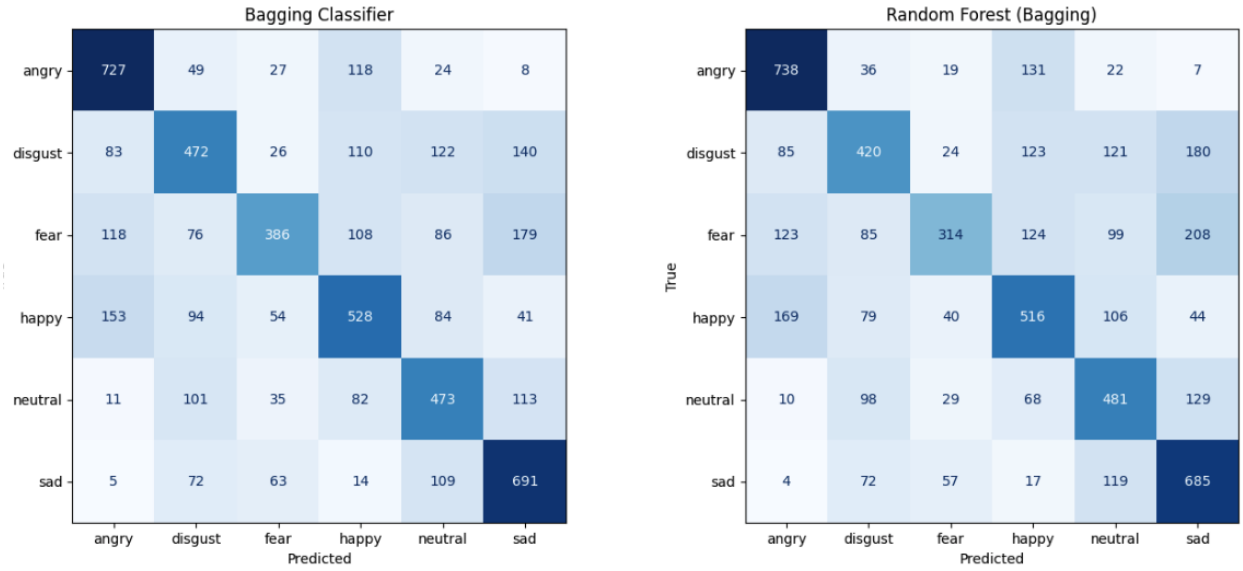


Figure 6 - Confusion Matrices for Top 2 Models

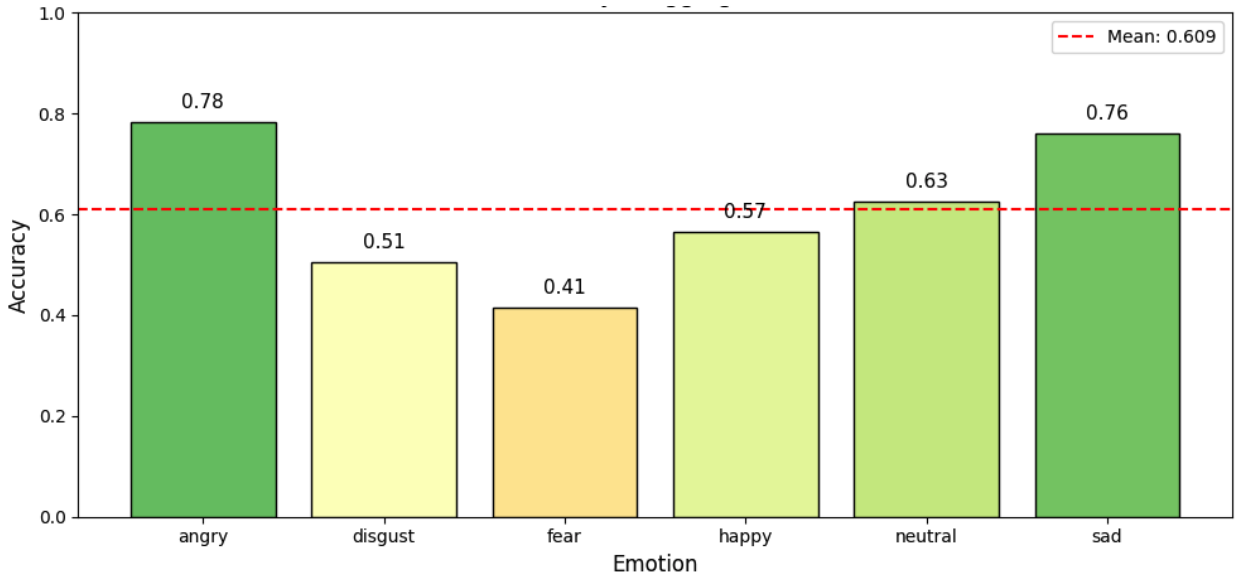


Figure 7 - Per-Emotion Accuracy (Bagging Classifier (Tuned))

This suggests that the limitation is not only the classifier, but also the feature representation and global decision strategy. Overcoming this ceiling requires **addressing the root causes of these confusions**. The **features need enhancement** to capture the dynamics that distinguish emotions with similar average characteristics. The **classification strategy needs sophistication** beyond single-model approaches, potentially combining multiple models that make different types of errors. These improvements are systematically explored in Part B of this report.

4. PART B: Systematic Improvements

The goal of Part B was to systematically and significantly improve upon the baseline established in Part A. Through a series of methodical experiments in feature engineering, hyperparameter tuning and ensemble architecture, we achieved a new performance with our final model, `stacking_diverse`, which reached **82.22% accuracy**. This section details the key factors behind this success and the experiments that led to our conclusion.

Three key observations emerged from Part A that became the foundation for our experimentation:

Observation 1: Fear-Sad, Angry-Happy, Disgust-Sad Confusion Problem

Examining the confusion matrix from our best baseline model (Bagging, 60.9%), we discovered that errors were not random – they clustered around specific emotion pairs. Fear was misclassified as sad most of the time, angry was misclassified as happy, and disgust was misclassified as sad. This pattern revealed that certain emotions are acoustically similar: fear and sad both involve quiet, hesitant speech; angry and happy both involve loud, energetic speech. We need to design features or classifiers specifically target these confused pairs.

Observation 2: Static Features Lose Temporal Information

Our baseline features only captured the average values across the whole 2.5-second clip. This caused different patterns to look the same. For example, someone whose pitch rises from 100 to 200 Hz and someone whose pitch falls from 200 to 100 Hz both end up with the same average pitch of 150 Hz. But these two patterns can signal very different emotions. That is when we realized we needed features that capture how the voice changes over time.

Observation 3: Different Algorithms Fail Differently

Bagging Classifier and Random Forest achieved 60.9% and 57.7% respectively, SVM achieved 56.2% and Naive Bayes achieved 23.1%. We noticed these algorithms made different errors**. SVM struggled with fear-sad distinctions**, while **Bagging Classifier/Random Forest struggled more with happy-angry distinctions**. Naive Bayes failed everywhere due to its independence assumption. This observation led us to hypothesize that **combining models** might allow their **strengths to compensate for each other's weaknesses**.

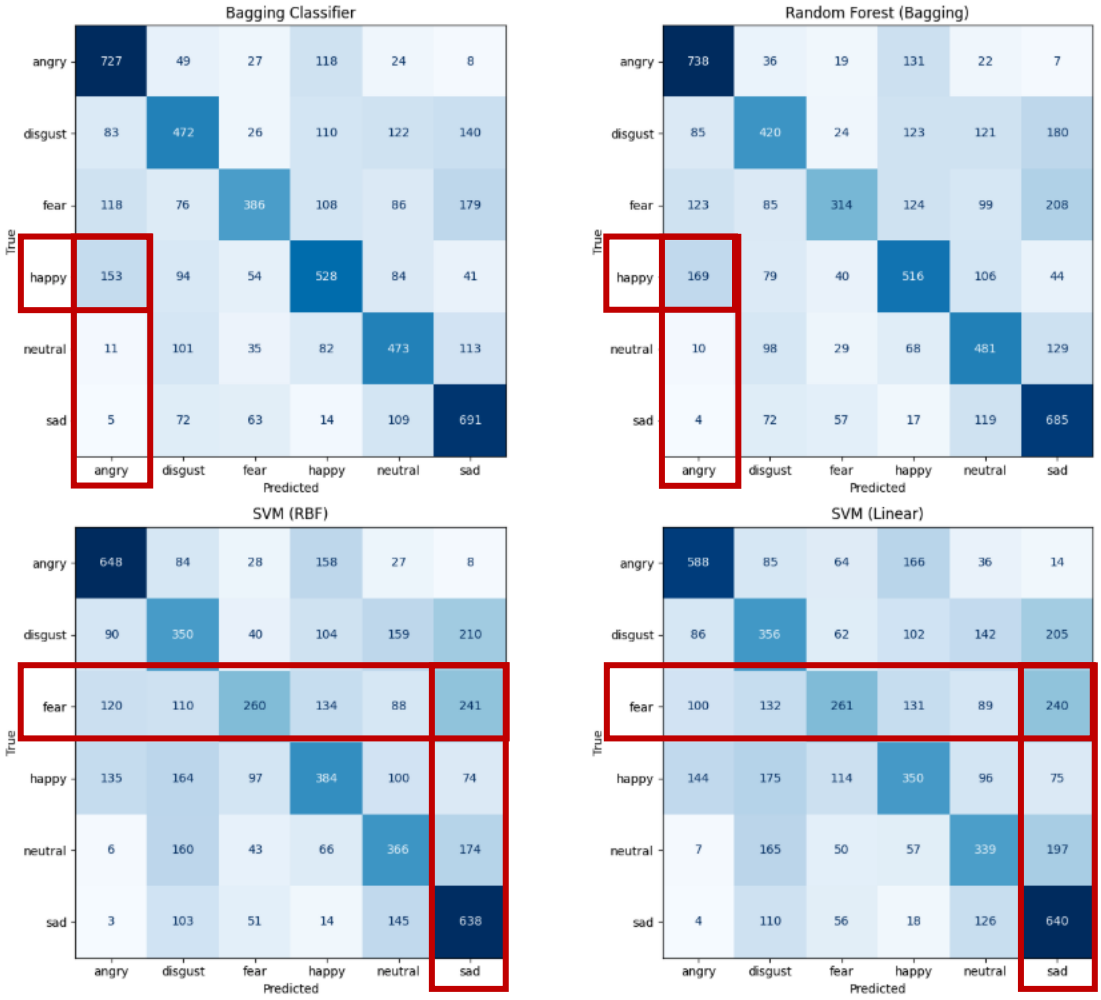


Figure 8 - Confusion Matrices for Top 4 Base Models

4.1 Key Sources of Performance Improvements

The leap in performance between Part A and Part B was the result of **(i) Enhanced Feature Engineering**, **(ii) Introducing Gradient Boosting Model, (iii) Systematic Hyperparameter Tuning** (beyond Part A).

These three factors work in synergy: the advanced features provide better data, Gradient Boosting and the other tuned base models are powerful enough to exploit that data, and the richer tuning process enables each model to sit at a near-optimal operating point.

Enhanced Feature Engineering – Adding New Features: Delta and Delta-Delta MFCCs (+40 features)

Delta features capture the rate of change of MFCCs over time. It tells us how the voice is changing.

Delta-Delta is the derivative of Delta. It captures the acceleration of change. A positive Delta-Delta value indicates that the voice is changing faster, such as when a speaker builds toward an emotional peak. A negative Delta-Delta value indicates that the rate of change is slowing down, such as when a speaker trails off at the end of a sad sentence.

High variance in Delta-Delta values across an utterance indicates erratic, unstable speech patterns characteristic of fear and anger, where the voice unpredictably accelerates and decelerates. Low variance indicates stable, controlled speech typical of neutral or calm states.

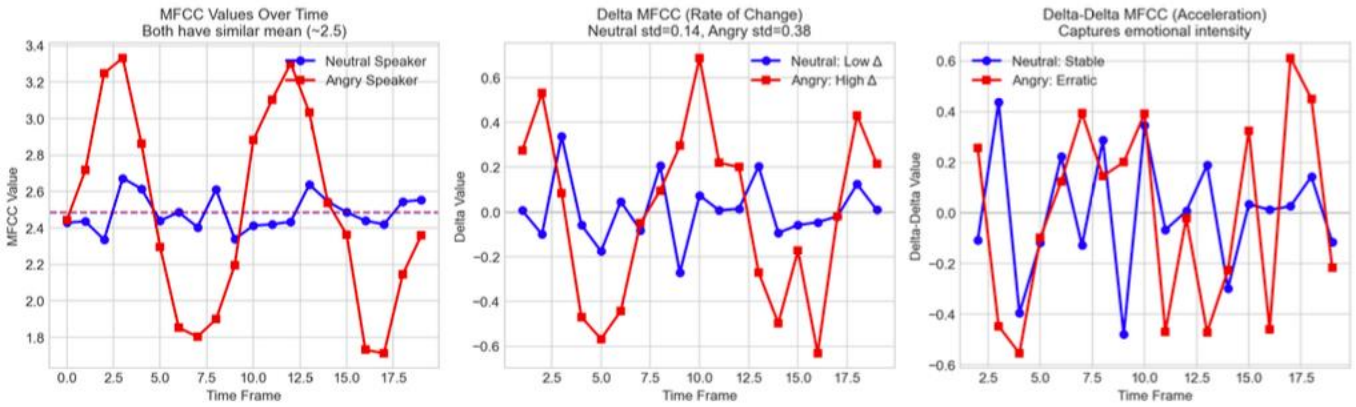


Figure 9 - MFCC Values, Delta MFCC, Delta-Delta MFCC

Enhanced Feature Engineering – Adding New Features: Prosodic Features (+8 features)

When spoken with a rising pitch that goes up at the end, it conveys a genuine question seeking confirmation. When spoken with a flat, unchanging pitch, it conveys disbelief or skepticism. When spoken with a sharp rise followed by a dramatic fall, it conveys sarcasm or irony. The words may be identical in all three cases, but the prosodic patterns completely change the emotional meaning.

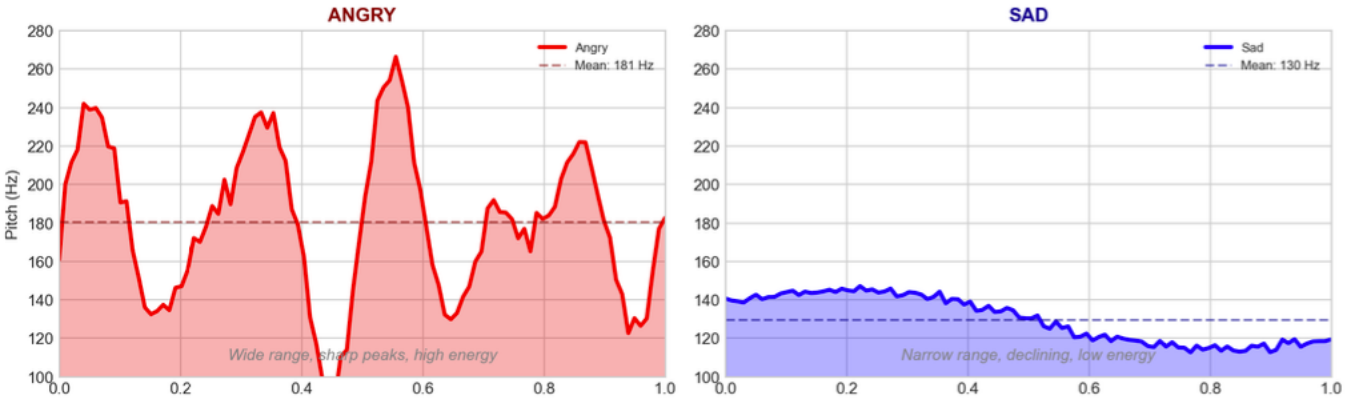


Figure 10 - Pitch Contours Angry - Sad "I can't believe it"

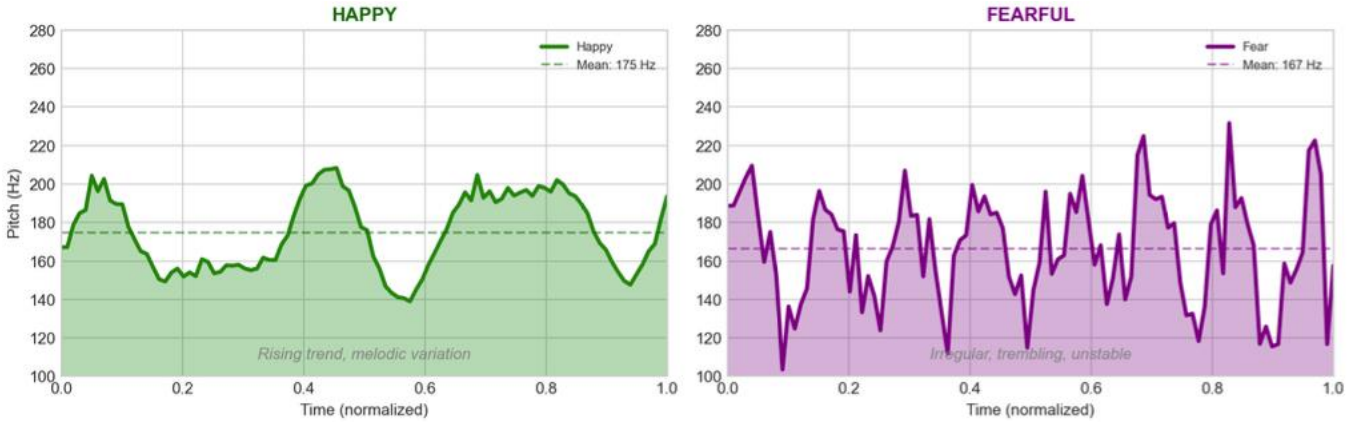


Figure 11 - Pitch Contours Happy - Fearful "I can't believe it"

Introducing Gradient Boosting Model – The best standalone model

The key challenge as identified earlier is distinguishing between emotions that are acoustically similar, such as ‘sadness’ and ‘disgust’, or ‘fear’ and ‘anger’. The **sequential error-correction** of Gradient Boosting is **perfect** for this. The initial trees in the model might learn to distinguish the easy cases (e.g. loud ‘anger’ vs quiet ‘neutral’). The subsequent trees then dedicate their capacity to the more ambiguous samples that were previously misclassified, learning the subtle, fine-grained differences in pitch, energy and spectral dynamics that separate these confusion emotions. This is why our analysis showed it was the best individual model (78.75%) for identifying all six emotions.

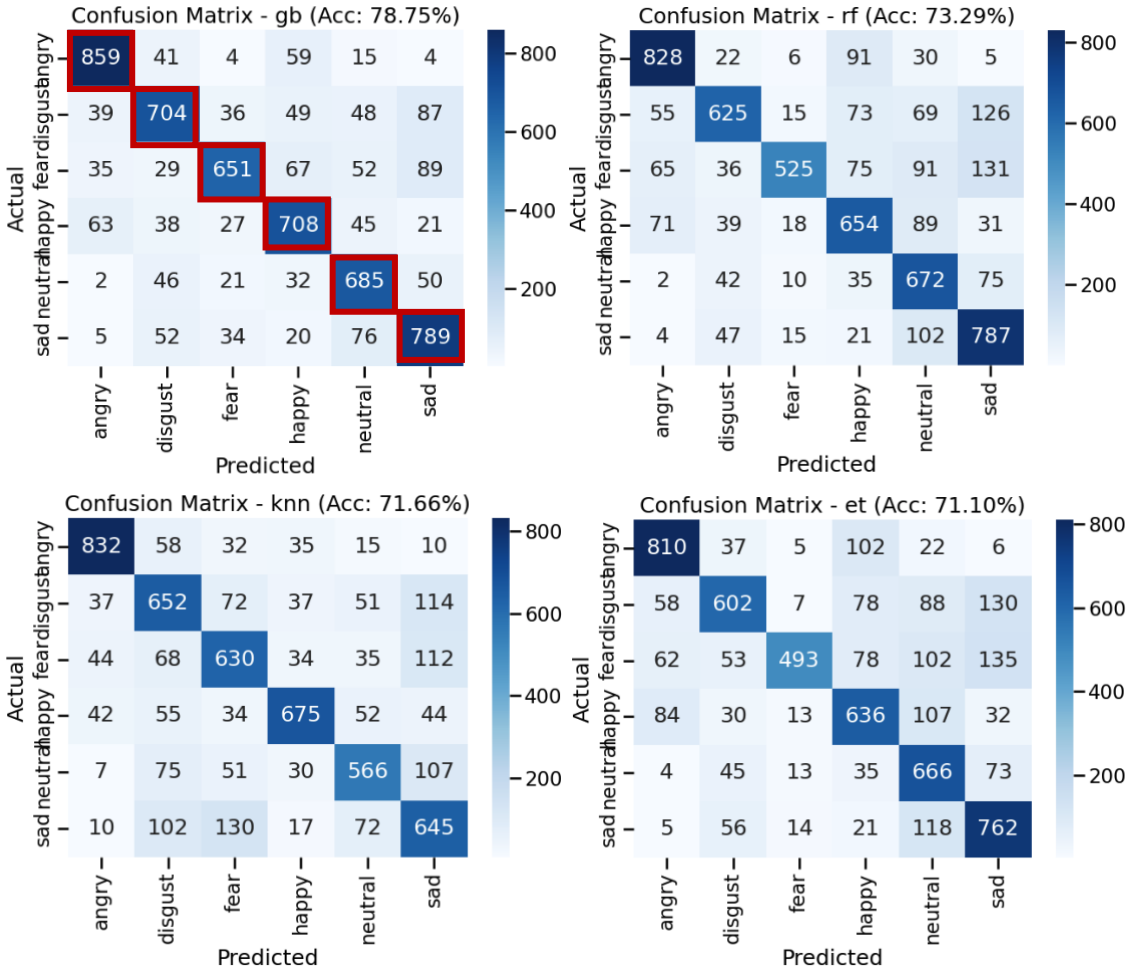


Figure 12 - Confusion Matrices for Top 4 Models

Hyperparameter Tuning (beyond Part A)

To ensure each base model performed at its peak, we conducted extensive hyperparameter tuning using `RandomizedSearchCV` with 5-fold cross-validation. We deliberately chose `RandomizedSearchCV` instead of a full `GridSearchCV` because running a dense grid with 5 folds across seven different models (each with several continuous or wide-range parameters) would have been computationally prohibitive. Random search lets us fix a reasonable number of iterations while still exploring a broad hyperparameter space, trading a small amount of optimality for speed.

The following table summarizes the key hyperparameters found for each of the core models:

Model	Hyperparameter	Best Value	Description
SVM	C	54.08	Smaller C means stronger regularization
	kernel	poly	The kernel function used to map data to a higher dimension
	gamma	0.0049	Kernel coefficient, defining the influence of a single training example
	degree	4	Degree of the polynomial kernel
Random Forest	n_estimators	463	Number of trees in the forest
	max_depth	26	Maximum depth of each tree
	max_features	sqrt	Number of features to consider when looking for the best split
	bootstrap	False	False means the whole bootstrap dataset is used.
KNeighbors	n_neighbors	2	Number of neighbors to use
	weights	distance	Weight function; 'distance' gives more weight to closer neighbors
ExtraTrees	n_estimators	60	Number of trees in the forest
	max_depth	14	Maximum depth of each tree
GradientBoosting	n_estimators	666	The number of boosting stages (trees) to perform
	learning_rate	0.205	Shrinks the contribution of each tree
	max_depth	9	The maximum depth of each individual tree
LogisticRegression	C	0.472	Inverse of regularization strength
	solver	newton-cholesky	Algorithm to use in the optimization problem
GaussianNB	var_smoothing	0.0085	Portion of the largest variance of all features added to variances for stability

4.2 Retraining Baseline Models with All Improvements

After adding the 48 new features, introduced Gradient Boosting and enhanced hyperparameter tuning, we retrained our models to evaluate the improvements.

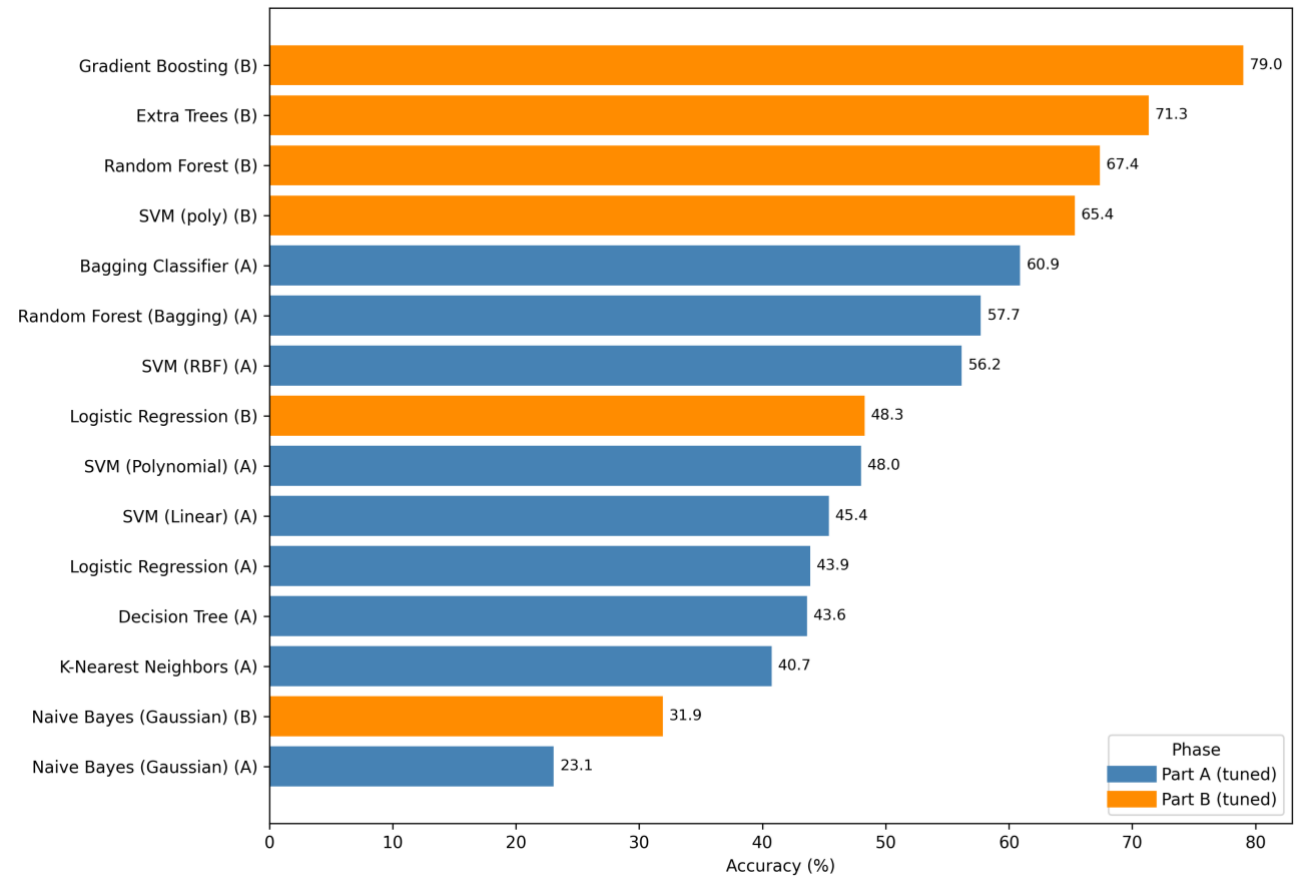


Figure 13 - Model Performance After Hyperparameter Tuning

It is clear from the results that the advanced features provided better data, the models are powerful enough to exploit the data and the richer tuning process enabled each model to sit at a near-optimal operating point.

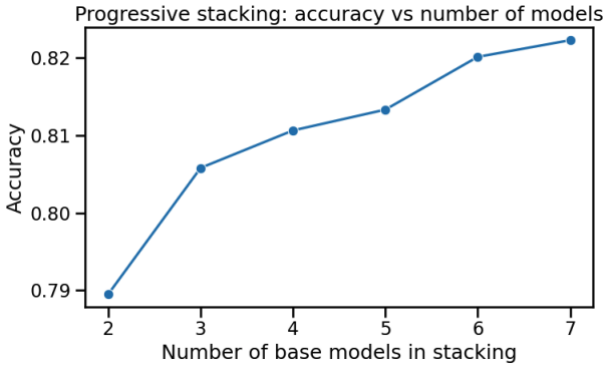
4.3 Finding the Optimal Ensemble – `stacking_diverse`

Our next experiment was to find the best way to combine seven powerful, tuned base models: SVM, RandomForest, Naïve Bayes, ExtraTrees, GradientBoosting, KNN and LogisticRegression. Studies have shown that Stacking Classifiers could lead to superior performance compared to other traditional machine learning models and even deep learning methods (Prajapati, 2025). Using sklearn's `StackingClassifier` + `LogisticRegression` Meta-learner therefore offers to increase the diversity of the base models and potentially lead to better performance.

The Stacking Classifier operates in two phases: first, it trains all base models on the complete dataset; then, it trains the meta-classifier on the predictions of these base models. This approach aims to learn the optimal way to combine diverse model predictions, potentially enhancing overall emotion recognition performance (Prajapati, 2025).

We started with our strongest individual model, `GradientBoosting`, and iteratively added the next-best-performing models one by one. The results from our notebook were clear and consistent: accuracy improved with each model we added. This demonstrates that each model, **even the weakest one (Naïve Bayes)**, brought a **unique "perspective"** that helped the stacking classifier's meta-learner make **better decisions, reducing overall error**.

This process resulted in the `stacking_diverse` model, which includes all seven base estimators. This single-layer ensemble achieved the highest accuracy of all our experiments and could be attributed to the ability to leverage the strengths of multiple base models and learn optimal combinations of their predictions, proving the value of maximizing algorithmic diversity.



Iter	Models	Accuracy	F1 Wgt
1	gb,rf	78.95%	78.91%
2	gb,rf,knn	80.58%	80.55%
3	gb,rf,knn,et	81.06%	81.04%
4	gb,rf,knn,et,svm	81.33%	81.32%
5	gb,rf,knn,et,svm,lr	82.01%	81.99%
6	**gb,rf,knn,et,svm,lr,nb**	**82.22%**	**82.20%**

a

4.4 Can we do better with an Ultra Ensemble (Ensemble of Ensembles)?

Our final experiment tested the hypothesis that we could improve performance even further by creating a second layer of ensembling. We created several "Ultra Ensemble" variants that combined the predictions of our best single model (`stacking_diverse`) with other strong component models (like `stacking_selected` where we select the top 100 most important features, and a standalone `gradient_boosting`).

However, our empirical results disproved this hypothesis. Our analysis showed that every combination that included a second layer of ensembling performed worse than the `stacking_diverse` model on its own. The best of these multi-layer ensembles achieved 81.49% accuracy, a drop from the 82.22% achieved by `stacking_diverse`. This is a critical insight: adding more complexity is not always better. In this case, the second layer of averaging likely diluted the strong, confident predictions from the already highly-optimized `stacking_diverse` model, introducing more noise than signal and ultimately harming performance.

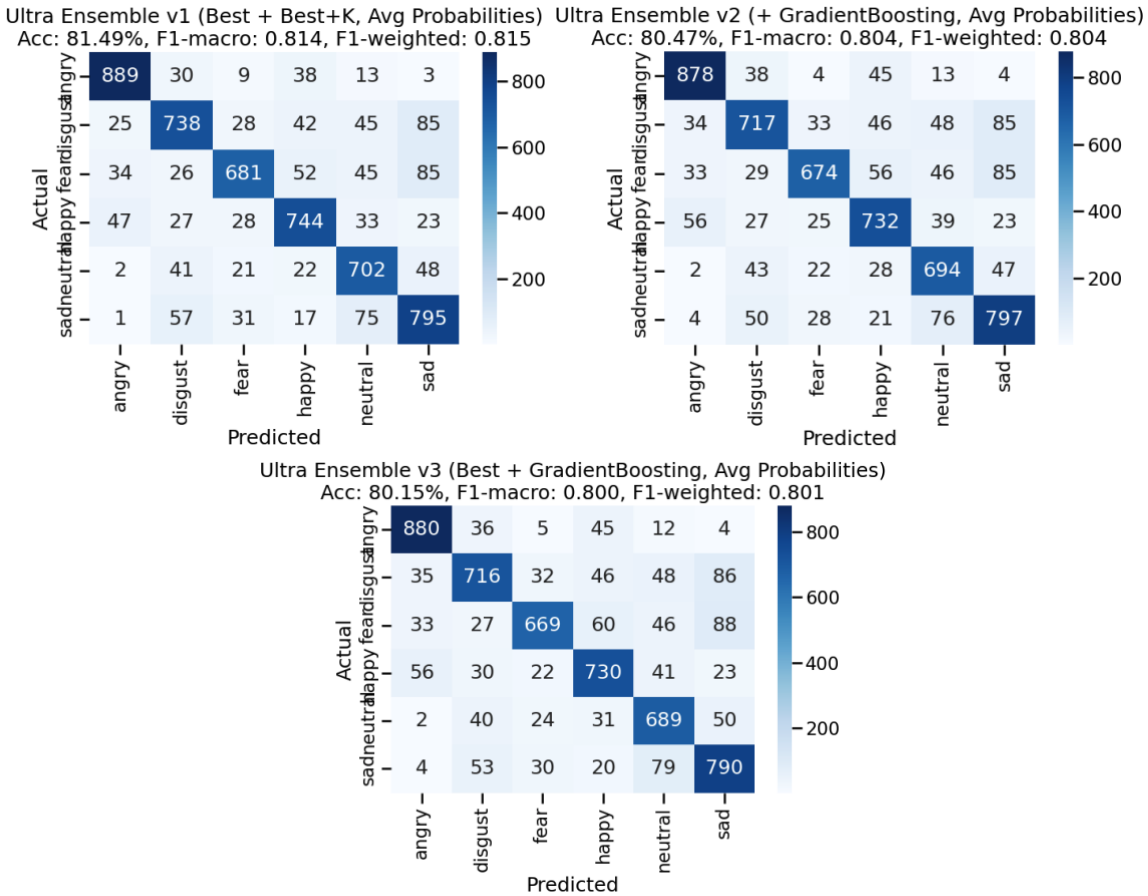


Figure 14 - Ultra Ensemble

4.5 Overall Conclusion

Our systematic investigation in Part B yielded two insightful results. First, we confirmed that **combining advanced feature engineering** with **systematic hyperparameter tuning** provides a **massive leap in performance** over a baseline approach.

Second, we analysed what happens when progressively adding models into the stack. Starting from our strongest individual performer, `GradientBoosting`, we added the next-best models one at a time. The trend in our results was clear and repeatable: accuracy improved with each additional model. Even the **weakest performer**, Naïve Bayes, **contributed a distinct perspective**, enabling the meta-learner to make more accurate final predictions. This supports the idea that **model diversity**, not just individual strength, is a **major driver of stacking performance**.

Third, our experiments in ensemble architecture revealed a crucial insight: a single-layer `StackingClassifier` (`stacking_diverse`) containing a wide variety of tuned base models was the optimal design. Our hypothesis that a more complex, multi-layer ensemble would be superior was empirically disproven, as these models consistently underperformed relative to the simpler `stacking_diverse` model.

Therefore, the final recommended model from this project is the `stacking_diverse` classifier, which stands as our highest-performing model at 82.22% accuracy.

5. Developing a Production-Ready Machine Learning Application

The following section details the project source code, application architecture, SDLC and DevOps/MLOps flow used for deploying the model and application for project demonstration. We have adopted production-grade standards and practices for the entire software development and delivery lifecycle. All the classroom concepts and tooling has been applied and implemented in our application build.

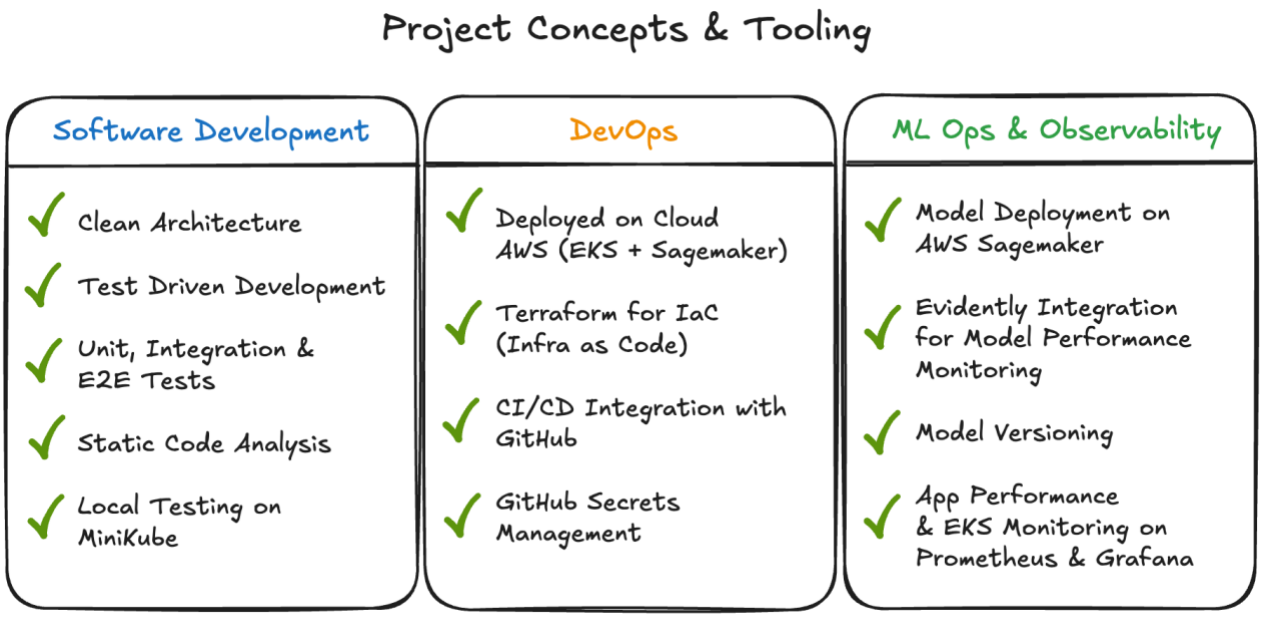


Figure 15 Classroom Concepts & Tooling applied in this project

5.1 Application Overview

The ML Speech Emotion Recognition (SER) application was developed as a proof-of-concept to establish the inference capabilities of our trained model and provide mechanisms to monitor model drifts and collect real-world data and labels for retraining. The application provides the following functionalities:

- Capability for users to upload audio file for emotion recognition
- Alternatively, users can perform a live recording of their speech as input
- Pre-trained model that predicts human emotion with confidence scores
- Audio Feature analysis
- Model Performance monitoring with Evidently integration
- Capability for users to provide feedback on prediction accuracy
- Application and infrastructure monitoring

The application is hosted in AWS and can be accessed over the internet at https://sagerstack.com/ml-ser/

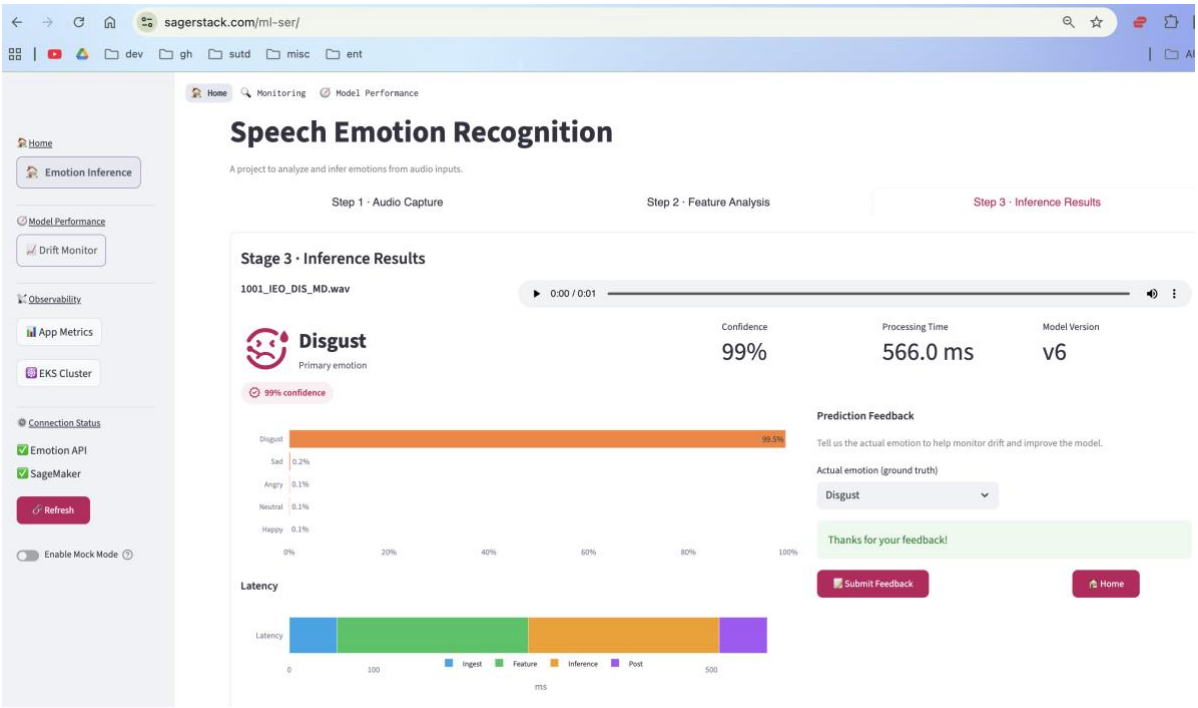


Figure 16 ML Speech Emotion Recognition App

5.2 Source Code / Steps to Reproduce Models and Plots

The complete source code for the ML-SER application is accessible at https://github.com/sagerstack/ml-speech-emotion-recognition.

```
## Jupyter Notebook & Model Training Setup
INSTRUCTIONS TO RETRAIN THE MODELS IN ITS ENTIRETY (TAKES 3-4HRS TO RUN)
*NOTE:* Due to the number of experiments that you have to run, each notebook will take 3-4 hours to run.

1. git clone https://github.com/sagerstack/ml-speech-emotion-recognition.git
2. Download CREMA-D Dataset:
```

```
  - Navigate to https://drive.google.com/drive/folders/1tvWeyxZM0bvKVhogRD1yCnwnpgOScJet?usp=share_link
  - Download `AudioWAV` folder in its entirety, and place the folder in `notebooks/AudioWAV`.
  - Alternatively, you may download CREMA-D Dataset from: https://www.kaggle.com/datasets/ejlok1/cremad
  - Ensure you unzip the audio files into `notebooks/AudioWAV` directory.
3. Install required python packages: `pip install -r requirements.txt`
4. Open `PART A - Baseline Models.ipynb` or `PART B - Systematic Improvements.ipynb` in Jupyter Notebook.
5. Run all cells in order (Kernel → Restart & Run All).
6. Training results and plots will be displayed at the end.

## ML Application Setup For Local Inference & Testing
- Download the v6 model and reference_dataset from [Google
Drive](https://drive.google.com/drive/folders/1p75TDFogCS5wt4x7VrA1JJcoVBDXTIfh?usp=drive_link)
- Next, follow the setup and deployment instructions in [user-guide-local-setup.md](docs/user-guides/user-guide-local-
setup.md).
```

The following is a high-level project structure of the repository

```
ml-speech-emotion-recognition/
├── backend/          # FastAPI service, monitoring, models, tests
├── frontend/         # Streamlit UI for inference, metrics, monitoring
├── deployment/       # Docker, Kubernetes, monitoring, infrastructure assets
├── notebooks/        # Jupyter notebooks for training/experiments
├── data/             # Local datasets (gitignored)
├── scripts/          # Utility scripts (upload, deploy, Terraform helpers)
├── docs/             # Architecture, operations, user guides
├── specs/            # Feature specs and user stories
├── reports/          # Project artifacts
└── README.md         # Local stack composition
```

5.3 Application Setup & Deployment Guide

For local deployment and running inference on your machine, please refer to README.md located in the root of the project repository in GitHub and follow the link to *docs/user-guides/user-guide-local-setup.md* for specific deployment instructions. Using the minikube deployment to spin up a local Kubernetes cluster will launch the complete application stack including monitoring capabilities. A typical output of the minikube deployment script (provided) is shown below.

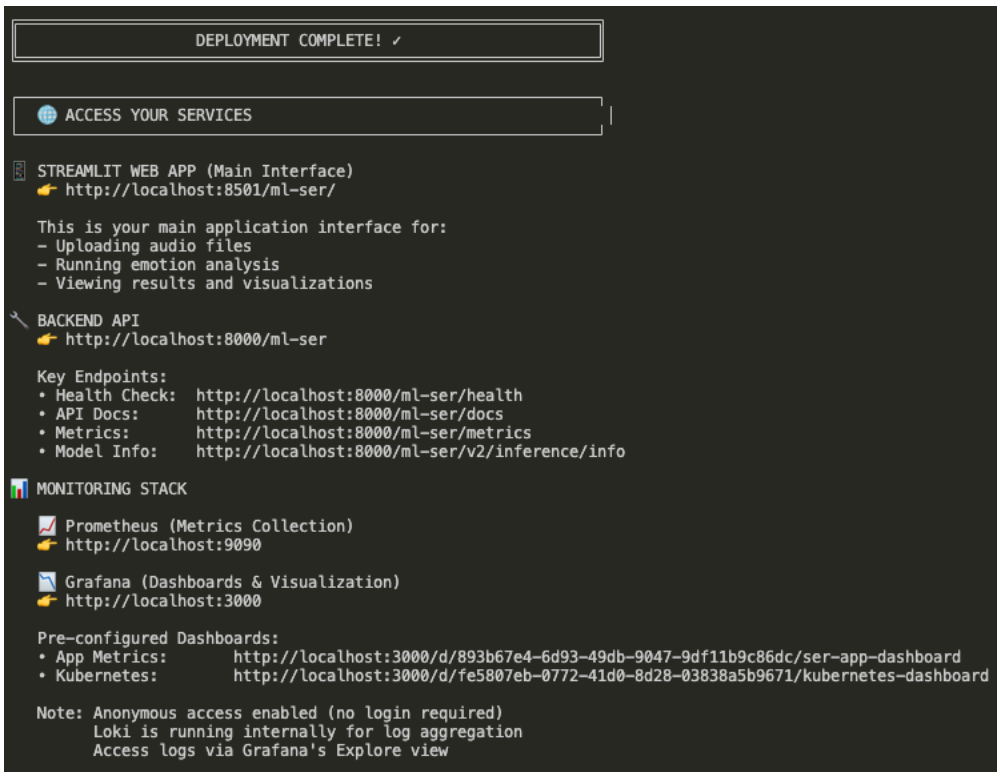


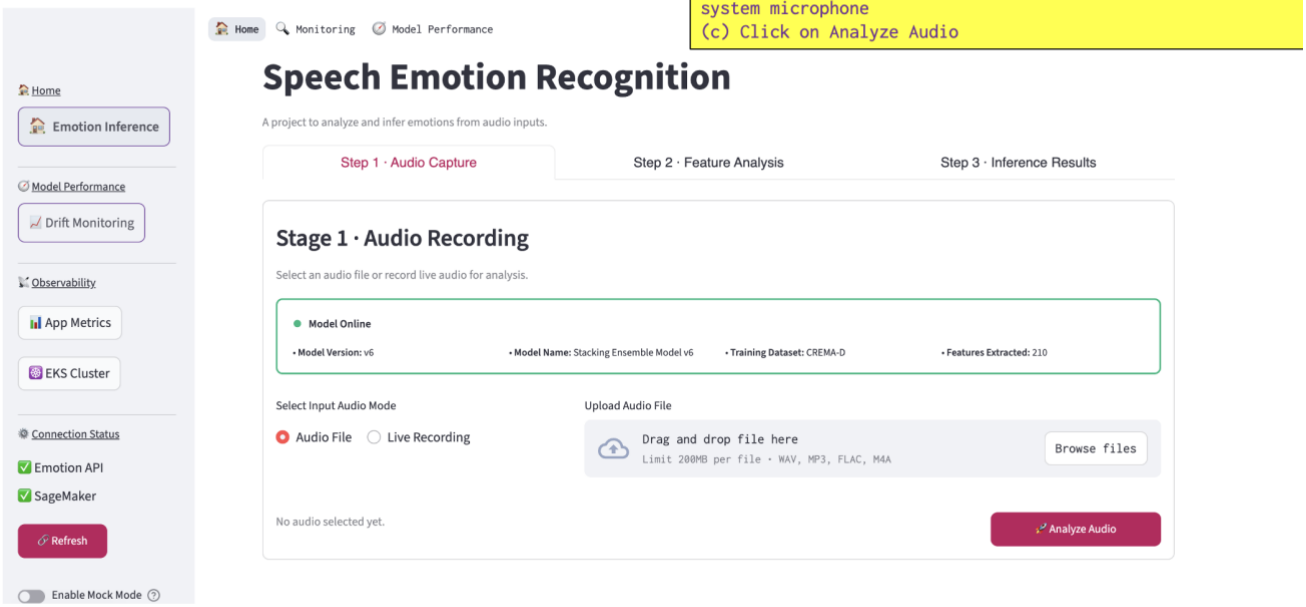
Figure 17 Local Deployment

Once local deployment steps are completed, please access the application via the following URLs-

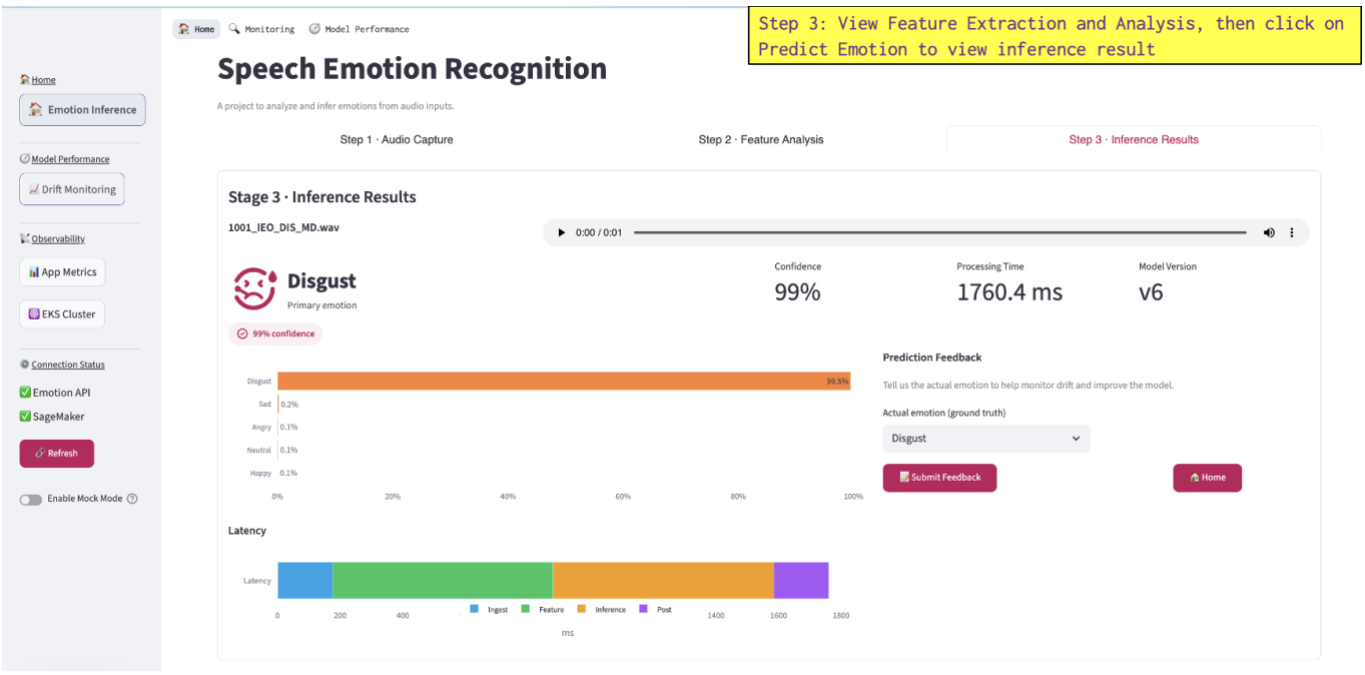
Streamlit Web App	*http://localhost:8501/ml-ser/*
Emotion API Backend	*http://localhost:8000/ml-ser/docs*
Model Info	*http://localhost:8000/ml-ser/v2/inference/info*
Monitoring Stack (Grafana)	App Metrics: *http://localhost:3000/d/893b67e4-6d93-49db-9047-9df11b9c86dc/ser-app-dashboard* Kubernetes: *http://localhost:3000/d/fe5807eb-0772-41d0-8d28-03838a5b9671/kubernetes-dashboard*

Application User Guide

Step 1: Provide audio input by either
(a) Loading an audio file (*.wav)
(b)Live recording of your speech by granting access to the system microphone
(c) Click on Analyze Audio



Step 2: View Feature Extraction and Analysis, then click on Predict Emotion to view inference result



5.4 Application Architecture

Container Diagram

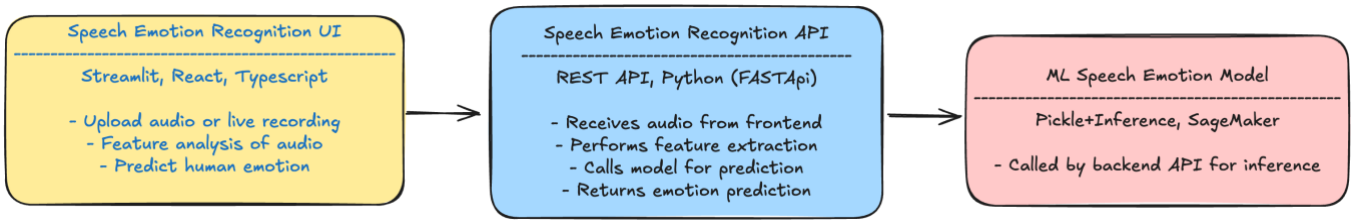


Figure 1818 Container Diagram outlining the major application components and technology stacks used

The frontend is developed using Streamlit and is primarily responsible for user interaction, inference results and analytics. It is a light-weight frontend, focuses purely on rendering the information received from the backend APIs and the model.

Backend is a FastAPI application hosting RESTful API endpoints that provide the integration with the pre-trained model.

The pre-trained models are the source of inference and deriving accuracy either hosted locally via pickle files mounted on the EKS volumes for local development and testing. For production, the same models are packaged and uploaded to AWS S3 and deployed in a custom sklearn container and accessed via Sagemaker endpoint.

Deployment Diagram

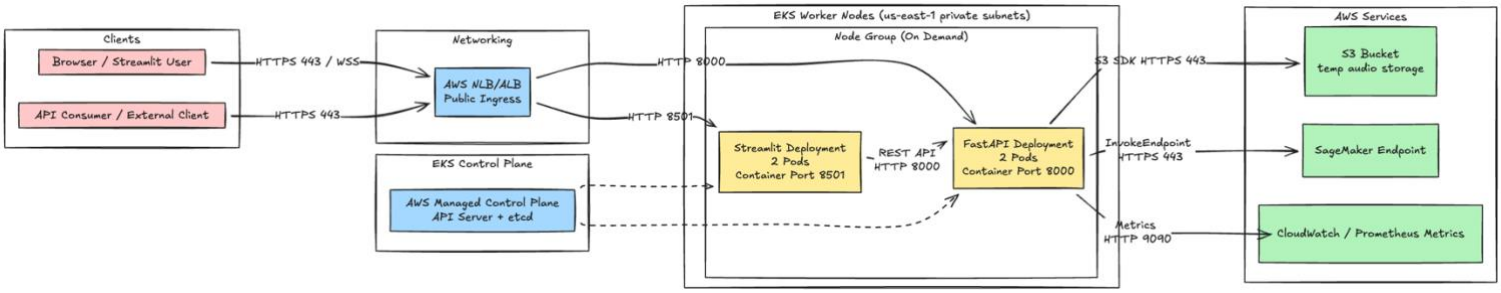


Figure 1919 Deployment Diagram for ML SER Application hosted in AWS

The app is deployed in AWS. Given the nature of this project is for academic purposes, we have configured a single AWS environment that serves as production. App development and testing is done locally by replicating the same AWS setup via minikube (local k8s cluster)

The primary components of the application are the Streamlit frontend and Backend API, both are deployed on EKS. Each service has 2 pods for effective load balancing and redundancy setup. Session stickiness is enabled to support stateful management of model predictions and feedback. The ML model is uploaded to S3 and exposed via a custom sklearn container via AWS Sagemaker Endpoint. The Backend API integrates with the model via AWS Sagemaker endpoint for inference.

5.5 Software Development Lifecycle (SDLC)

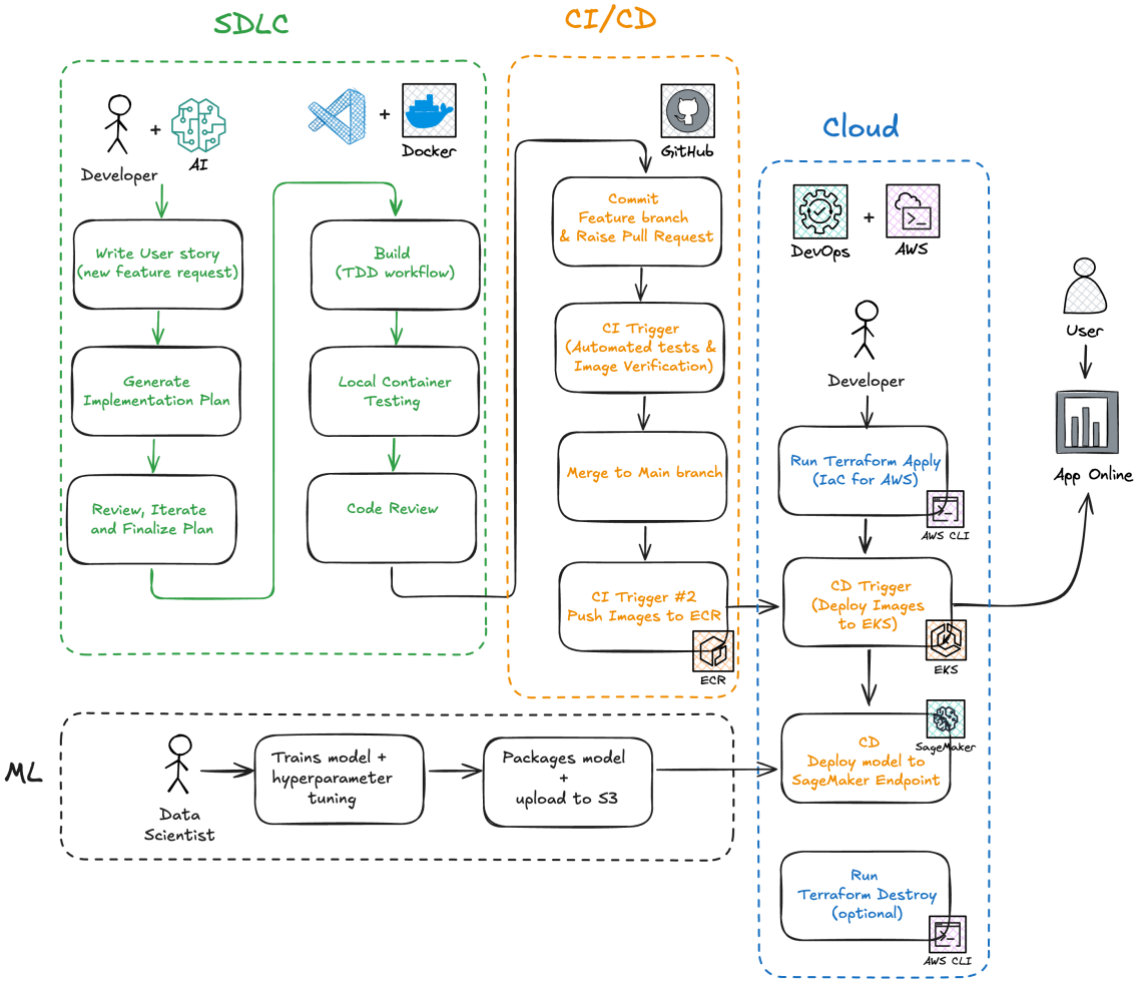


Figure 2020 A complete overview of the Machine learning and SDLC workflow for the ML Speech Emotion Recognition application.

This flow facilitated a clean separation of concerns between the Agile SDLC followed by Software engineers and the model training/evaluation conducted by Data Scientists. Data Scientists were required to be nimble and fast, and this pipeline allowed them to iteratively upload their trained models to AWS S3, ready for inference. Eventually, both the application and ML model changes were deployed to the cloud via the same CI/CD pipelines.

5.6 Development Methodology

We used Test Driven Development (TDD) methodology to develop the app. Requirements were created as user story artifacts. User stories were translated into technical implementation plans and then developed. We used AI-assisted development cycles with code reviews done by human and used short-lived feature-branches to develop the app features. Each merge to main branch requires pull requests to ensure holistic code reviews, automated tests and container-image readiness checks are done before changes were merged into main.

5.7 Code Quality Standards

Both frontend and backend source code follows SOLID principles with a high regard for code quality, test coverage and static code linting and formatting rules. Backend adopts Clean Architecture to ensure segregation of domain, application and infrastructure concerns.

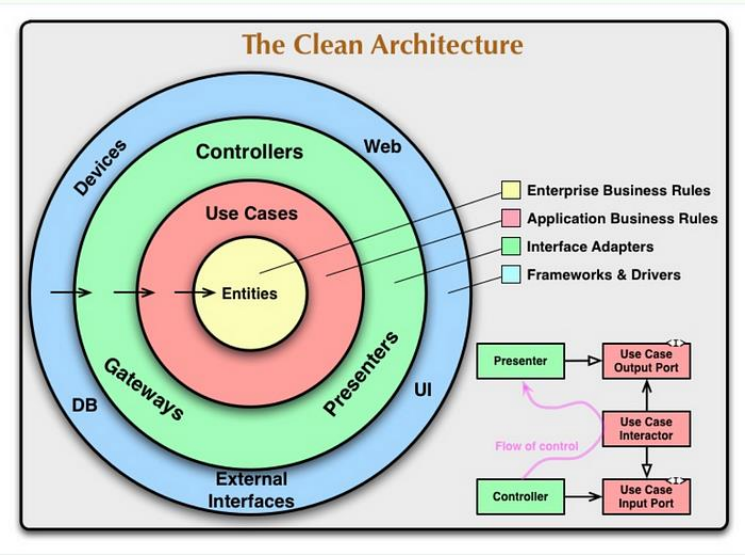


Figure 2121 Clean Architecture diagram referenced from https://blog.stackademic.com/the-clean-architecture-a-comprehensive-guide-for-beginners-7b52328a184d

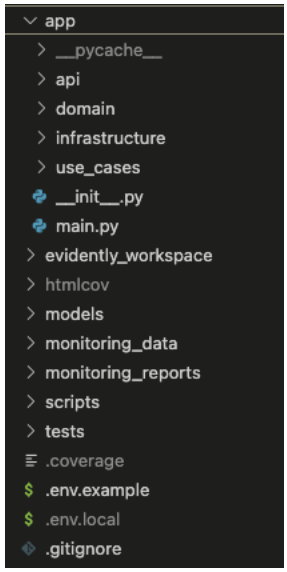


Figure 2222 SER Backend API repository structure implementing Clean Architecture

5.8 Model Versioning

A significant advantage of using clean architecture was the flexibility to manage different versions of the machine learning model that we trained and deployed. As the model was developed, trained and packaged, there were changes in how the application interacted with the model, since the feature extraction, scaling, prediction methods evolved over the course of the project. With clean architecture, we were able to deploy multiple versions of the model (in the infrastructure layer), while keeping the rest of the application abstracted from the model integration concerns.

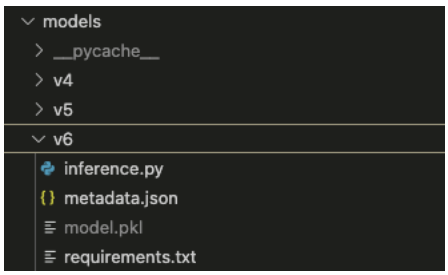


Figure 2323 Model versioning in source code (local inference)

Model versioning is also enforced during Sagemaker deployment, where every newly deployed endpoint is version controlled. Previous model versions remain available for quick rollbacks in production.

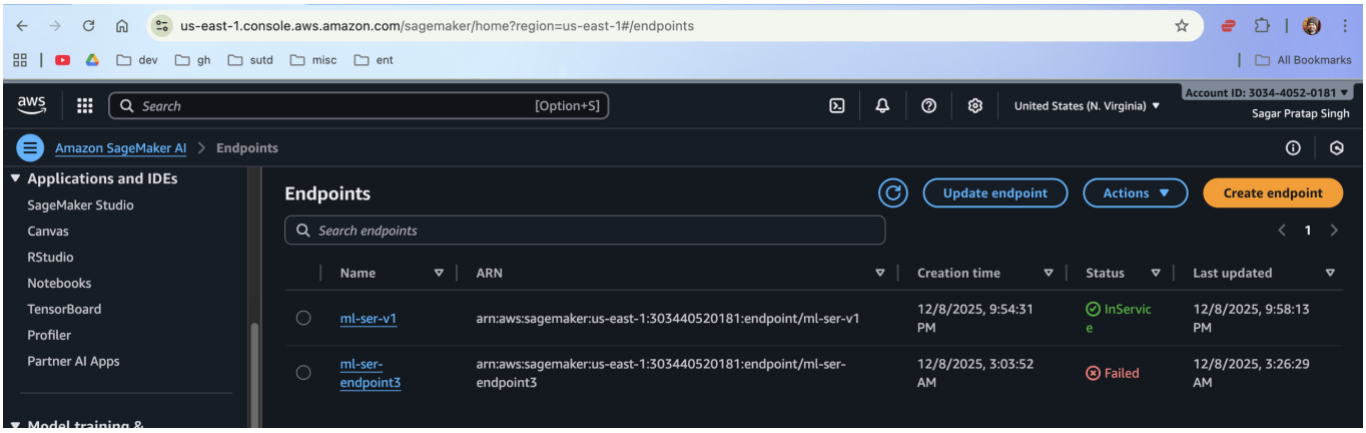


Figure 2424 Model versioning in AWS Sagemaker

5.9 Testing

We adopted the test pyramid strategy, with the following types of tests configured to thoroughly test the application features:

1. Unit tests
2. Integration tests
3. E2E tests

5.10 Infrastructure as Code (IaC)

For local development, we deployed and tested the app on local Kubernetes cluster using minikube. Docker images for both frontend and backend were pushed to Docker hub.

For production deployment, we used Terraform to spin up the AWS infrastructure whenever required- covering EKS, AWS Load Balancer, Sagemaker, Cloudfront, S3, etc. Given the cost implications, there are setup and teardown scripts in the repo that were used frequently to spin up, test and teardown the infrastructure.

```
module.eks.aws_eks_addon.this["vpc-cni"]: Creating...
module.eks.aws_eks_addon.this["coredns"]: Creating...
module.eks.aws_eks_addon.this["kube-proxy"]: Creating...
module.eks.aws_eks_addon.this["vpc-cni"]: Still creating... [00m10s elapsed]
module.eks.aws_eks_addon.this["kube-proxy"]: Still creating... [00m10s elapsed]
module.eks.aws_eks_addon.this["coredns"]: Still creating... [00m10s elapsed]
module.eks.aws_eks_addon.this["coredns"]: Creation complete after 16s [id=ml-speech-emotion-prod-eks:coredns]
module.eks.aws_eks_addon.this["kube-proxy"]: Still creating... [00m20s elapsed]
module.eks.aws_eks_addon.this["vpc-cni"]: Still creating... [00m20s elapsed]
module.eks.aws_eks_addon.this["kube-proxy"]: Creation complete after 26s [id=ml-speech-emotion-prod-eks:kube-proxy]
module.eks.aws_eks_addon.this["vpc-cni"]: Still creating... [00m30s elapsed]
module.eks.aws_eks_addon.this["vpc-cni"]: Creation complete after 37s [id=ml-speech-emotion-prod-eks:vpc-cni]
data.aws_eks_cluster_auth.this: Reading...
data.aws_eks_cluster.this: Reading...
aws_eks_addon.ebs_csi_driver: Creating...
data.aws_eks_cluster_auth.this: Read complete after 0s [id=ml-speech-emotion-prod-eks]
data.aws_eks_cluster.this: Read complete after 0s [id=ml-speech-emotion-prod-eks]
aws_eks_addon.ebs_csi_driver: Still creating... [00m10s elapsed]
aws_eks_addon.ebs_csi_driver: Still creating... [00m20s elapsed]
aws_eks_addon.ebs_csi_driver: Still creating... [00m30s elapsed]
aws_eks_addon.ebs_csi_driver: Still creating... [00m40s elapsed]
aws_eks_addon.ebs_csi_driver: Creation complete after 48s [id=ml-speech-emotion-prod-eks:aws-ebs-csi-driver]

Apply complete! Resources: 93 added, 0 changed, 0 destroyed.
```

Figure 2525 Terraform deployment in action

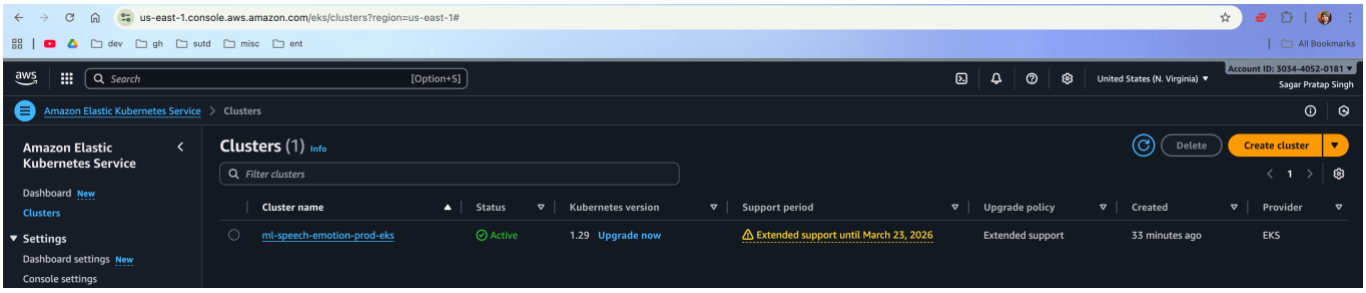


Figure 2626 EKS cluster setup automated by Terraform

5.11 Model Development Lifecycle

Models are developed by Data scientists and uploaded to S3 via an automated script that packages the pickle file and relevant metadata required for Sagemaker deployment.

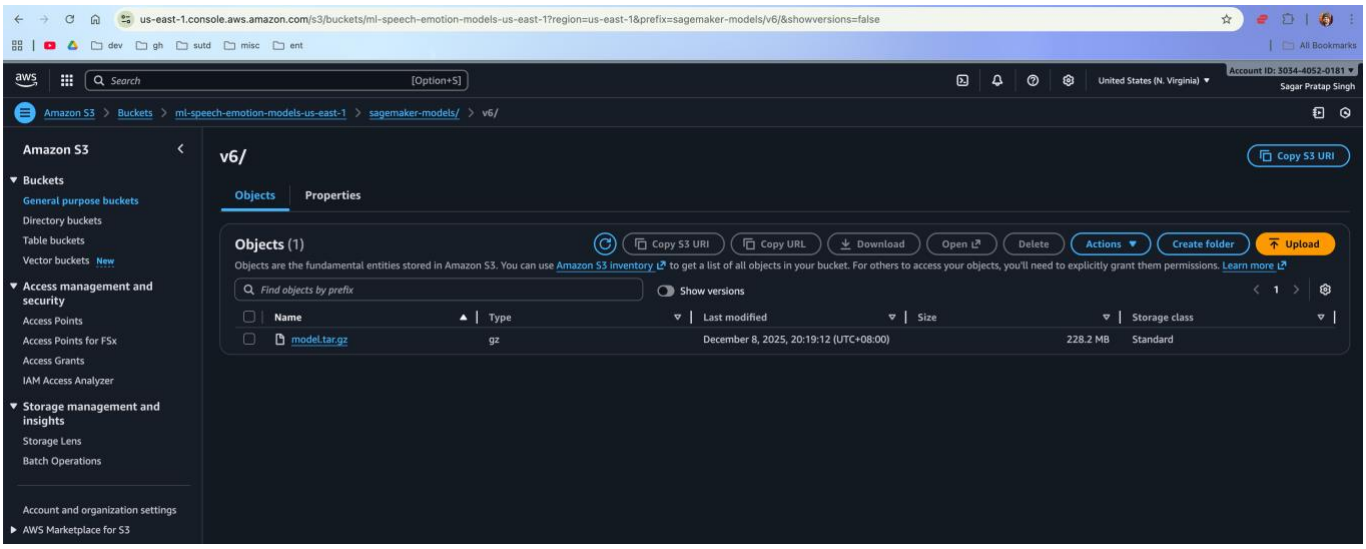


Figure 2727 Trained models are uploaded to S3 via Data scientists via a predefined script that packages the model for Sagemaker deployment

5.12 CICD

The following github actions were setup to facilitate the SDLC workflow:

Continuous Integration (CI)

CI is executed upon every pull request from feature branch to main, as well as a merge to main branch. The following actions are configured in the CI action:

- Run all tests using pytest. Fail if test coverage drops lower than 90%
- Use python libraries like **Black, Ruff** for formatting and linting checks
- Use **Mypy** for type checking
- Formatting checks of terraform scripts
- Build container images of both frontend and backend successfully
- Push container images to AWS ECR (Elastic Container Registry)
- Build and push custom sklearn container image to ECR

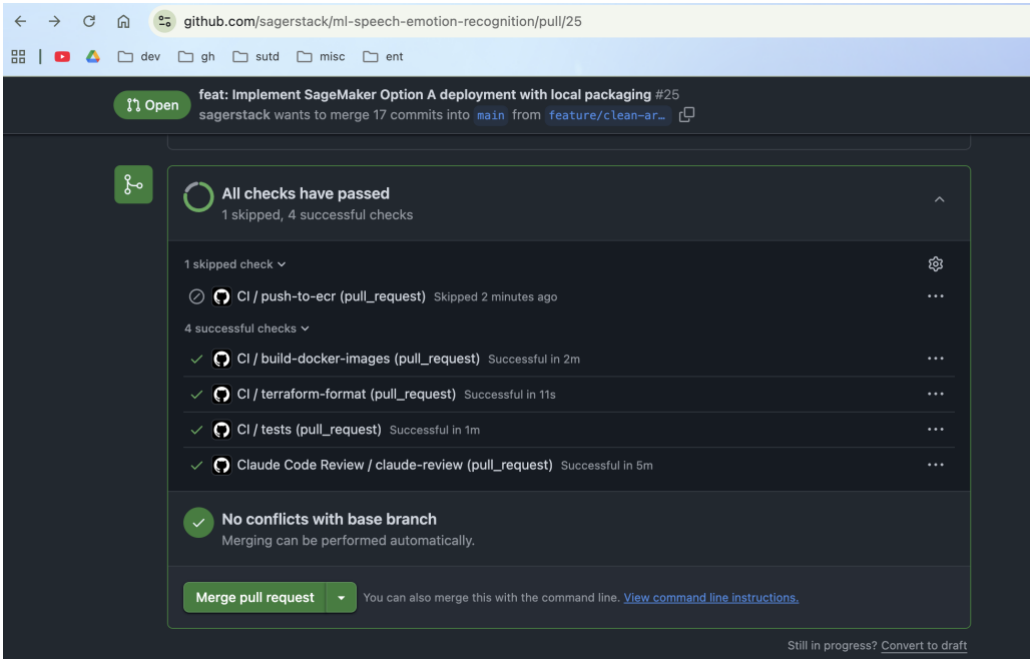


Figure 2828 CI github action automatically triggers when pull request is created

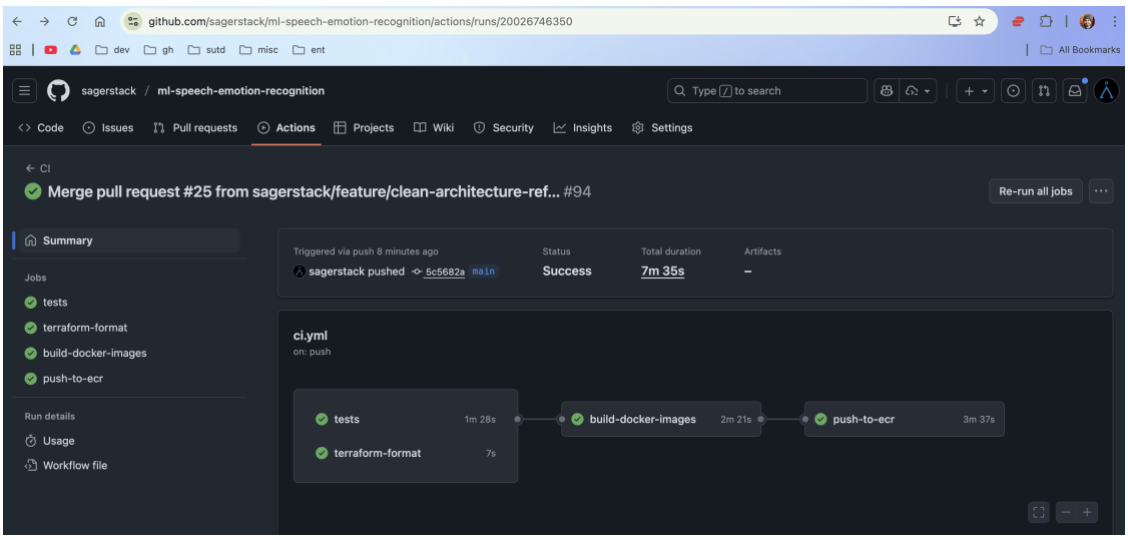


Figure 2929 CI github action pushes docker images to AWS Elastic Container Registry (ECR) when a pull request is merged to main branch

Continuous Deployment (CD)

CD action is on-demand and only executed from main branch (protected). This means that every new feature branch must go through the pull request and CI workflow to merge to main and generate a new image in AWS ECR. This is the mandatory prerequisite for a new deployment to be triggered via CD, ensuring code quality checks and automated test suite is passed before a new image is deployed to EKS.

CD step also deploys the latest model on AWS SageMaker for inference.

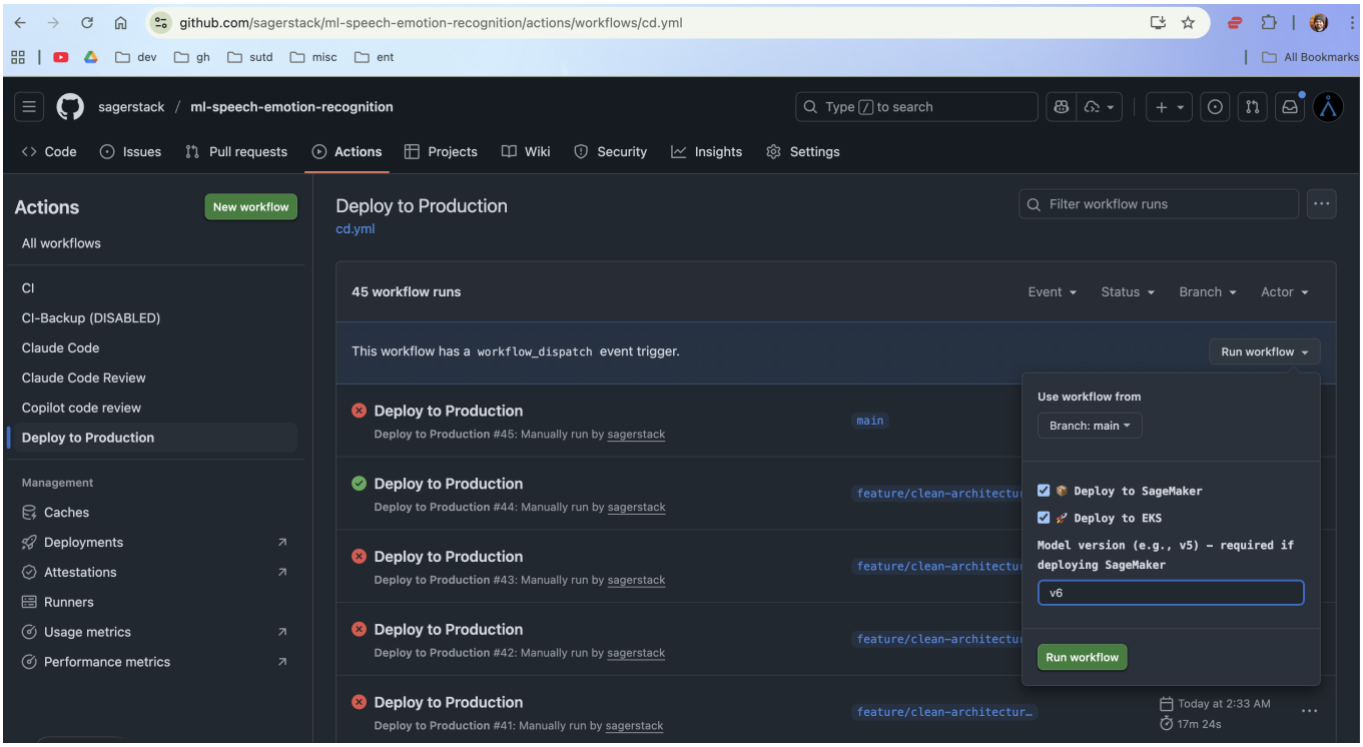


Figure 3030 CD action provides options to either deploy model to Sagemaker or deploy app to EKS, or both. It offers the flexibility needed for Data Scientists to run independently of typical SDLC

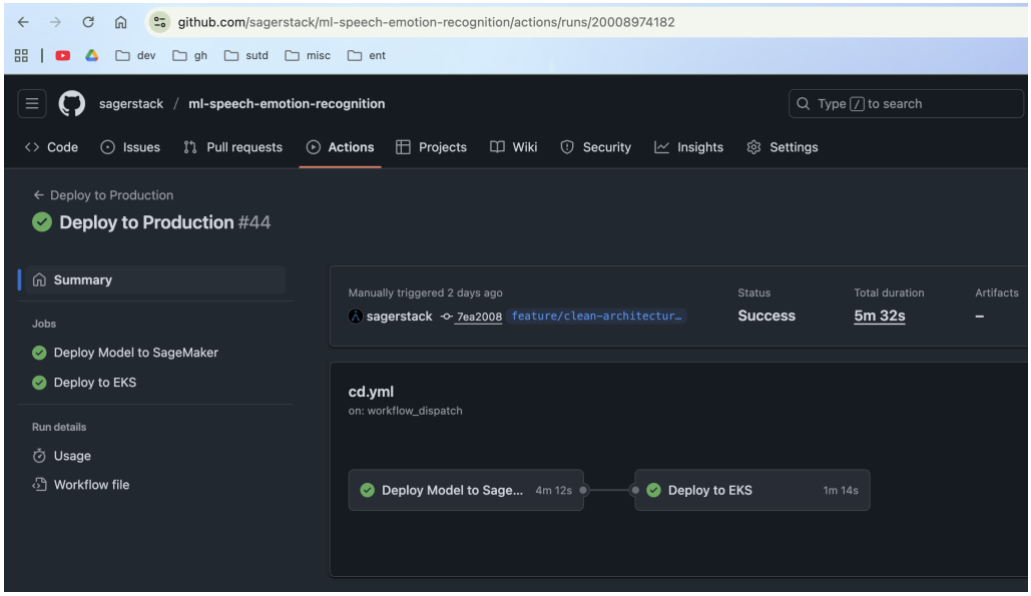


Figure 3131 CD github action deploys S3 model to a new inference endpoint in Sagemaker with version control

Agentic Code Reviews

Code reviews by AI agents are configured to automatically execute when a pull request is raised. This provides an additional data point for final code quality checks before PRs are merged, assuming the remaining checks are successful.

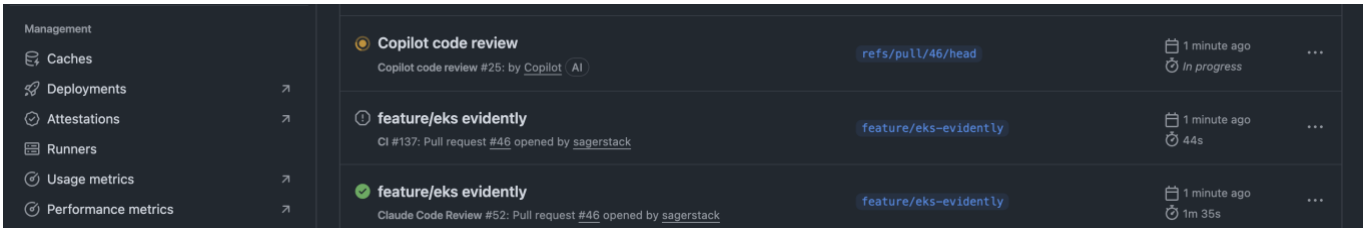


Figure 3232 Copilot code review shows in progress

5.13 Model Performance Monitoring

The SER Backend is integrated with Evidently to monitor the performance of our latest model by monitoring data drift, concept drift and label drift. It uses the features of the training dataset as a reference to evaluate data drift.

Additionally, the frontend provides a capability for users to confirm actual labels vs predicted labels on real-word data in production. With knowledge of the actual labels, Evidently can monitor concept drift (detecting degradation in accuracy, precision, recall, F1-score over time) and label drift (changes in the actual emotion distribution), enabling us to calculate real classification metrics and detect when the model's predictions are becoming less accurate compared to user-confirmed actual emotions.

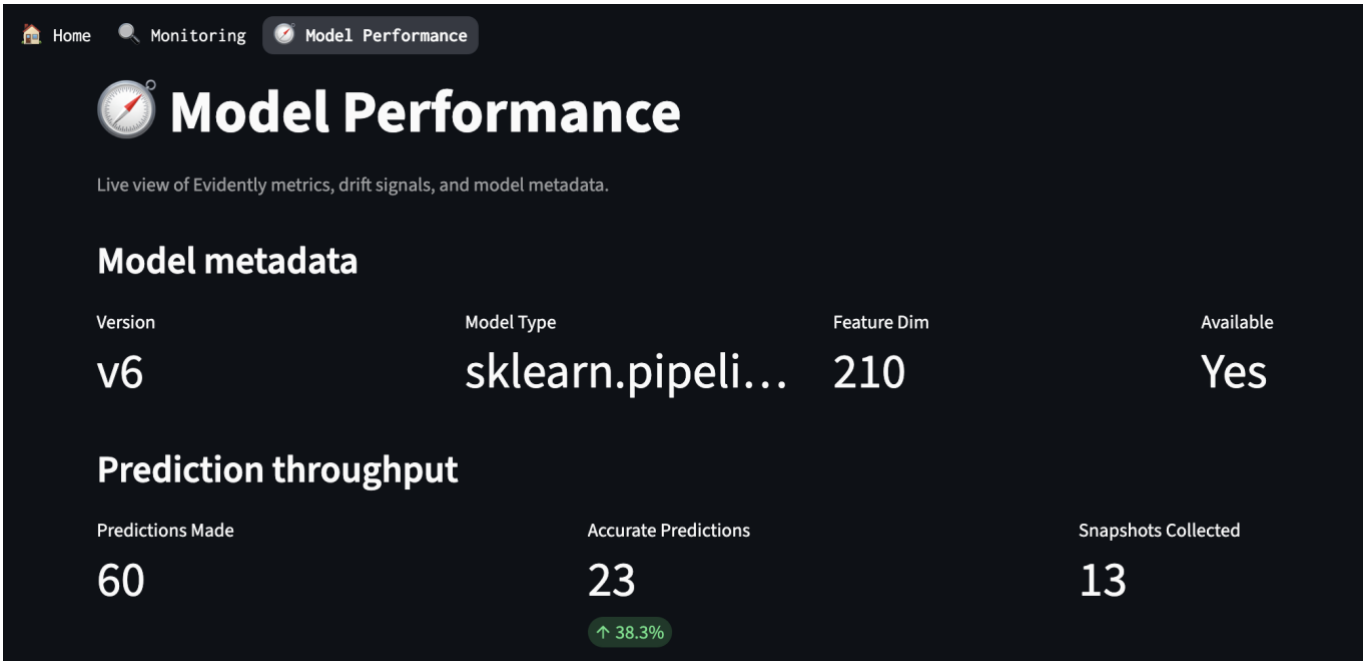


Figure 3333 Speech to Emotion Model Performance Monitoring using Evidently AI

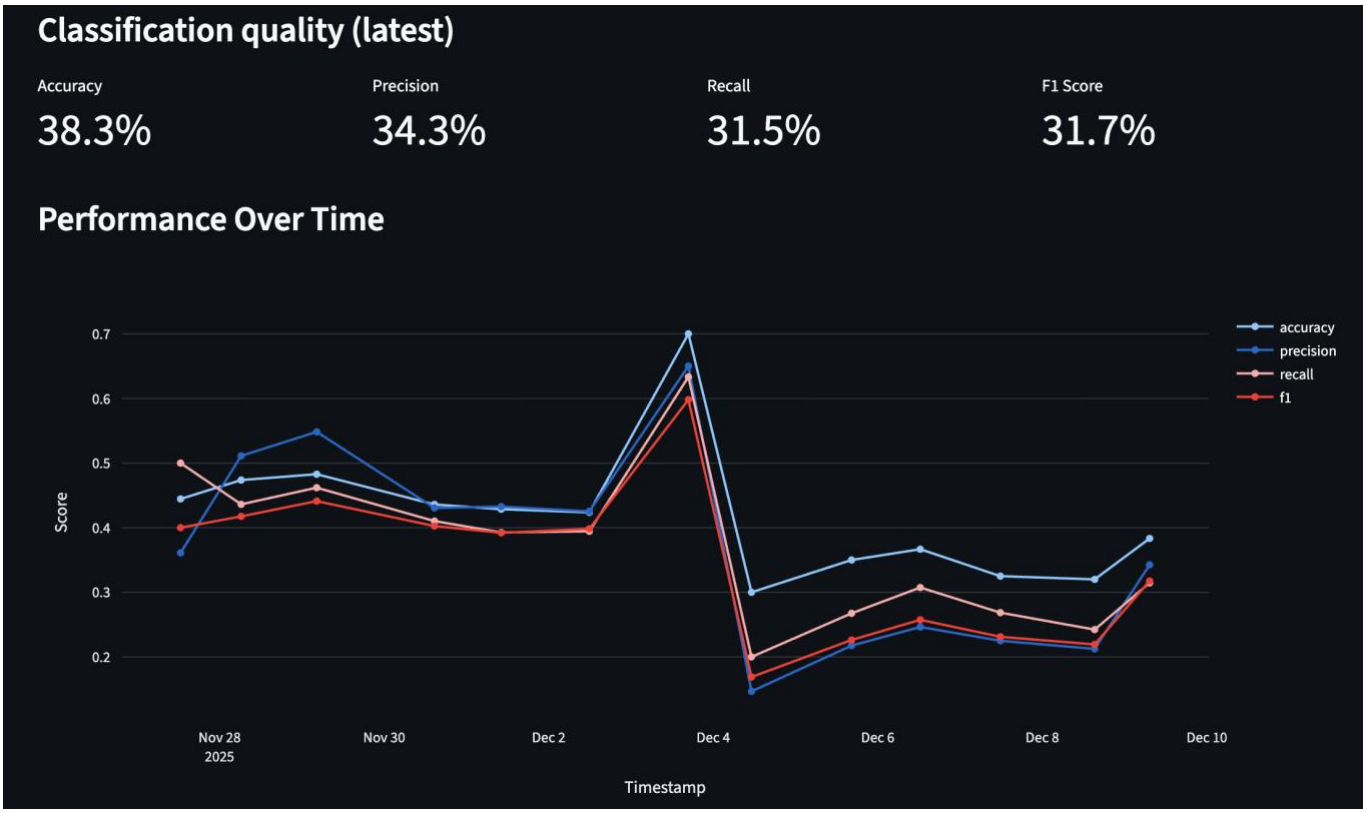


Figure 34 34 Classification quality metrics tracked over a 2-week period (Nov 28 - Dec 10, 2025) using Evidently AI. The chart shows accuracy, precision, recall, and F1 score trends for the speech emotion recognition model evaluated against RAVDESS audio samples.

Classification Performance shows significant volatility, with accuracy fluctuating between 20-70% over the monitoring period. The latest metrics indicate the model struggles to generalize to RAVDESS audio samples, performing well below the ~65% accuracy achieved on the CREMA-D test set.

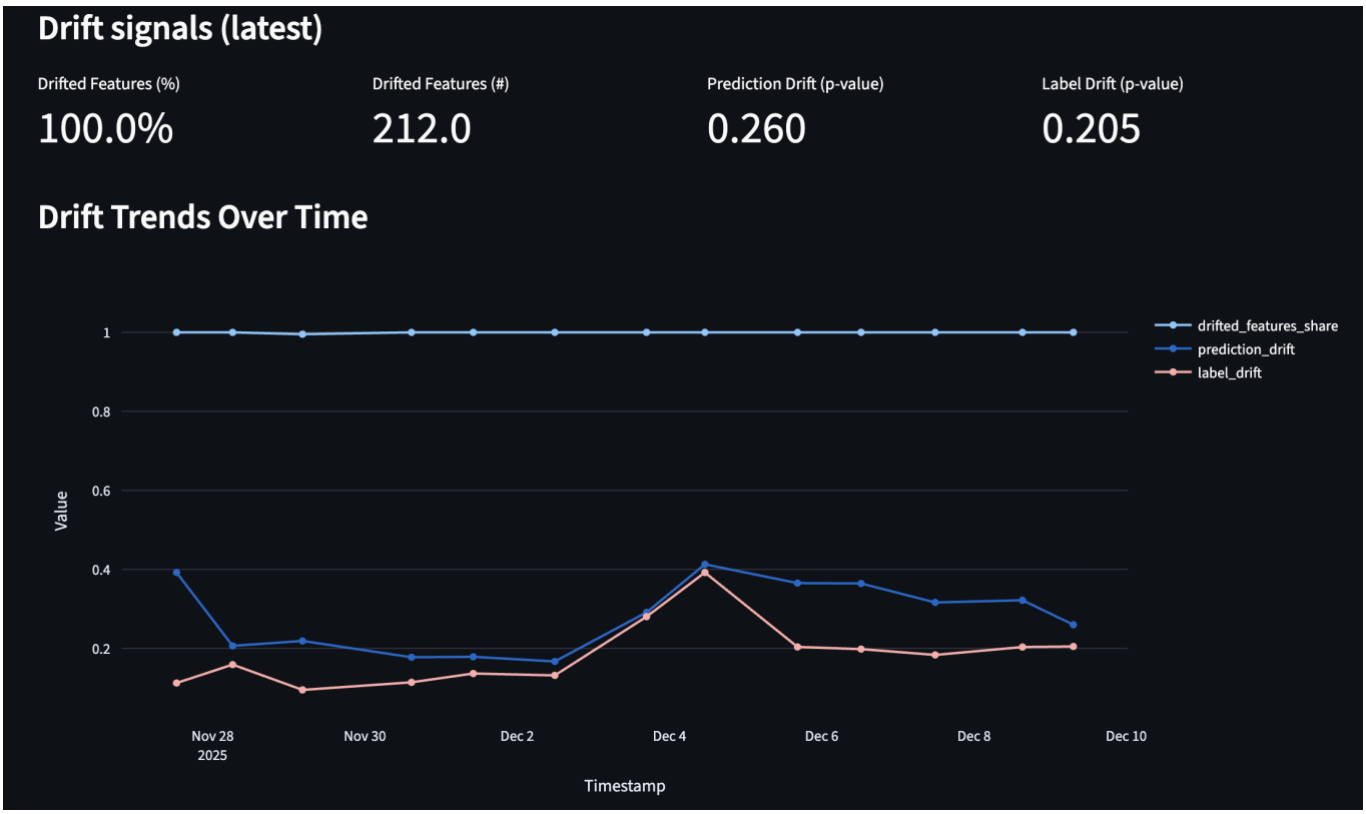


Figure 3535 Data Drift detection metrics tracked over a 2-week period (Nov 28 - Dec 10, 2025) using Evidently AI

Data Drift Detection reveals 100% feature drift across all 212 MFCC and spectral features, confirming substantial distribution differences between the CREMA-D reference data and RAVDESS production data. This is expected given the different recording conditions, actor demographics, and audio characteristics between the two datasets. Notably, prediction and label drift p-values (0.260 and 0.205) remain above the 0.05 threshold, indicating that while the underlying audio features have shifted significantly, the overall emotion class distributions remain statistically similar.

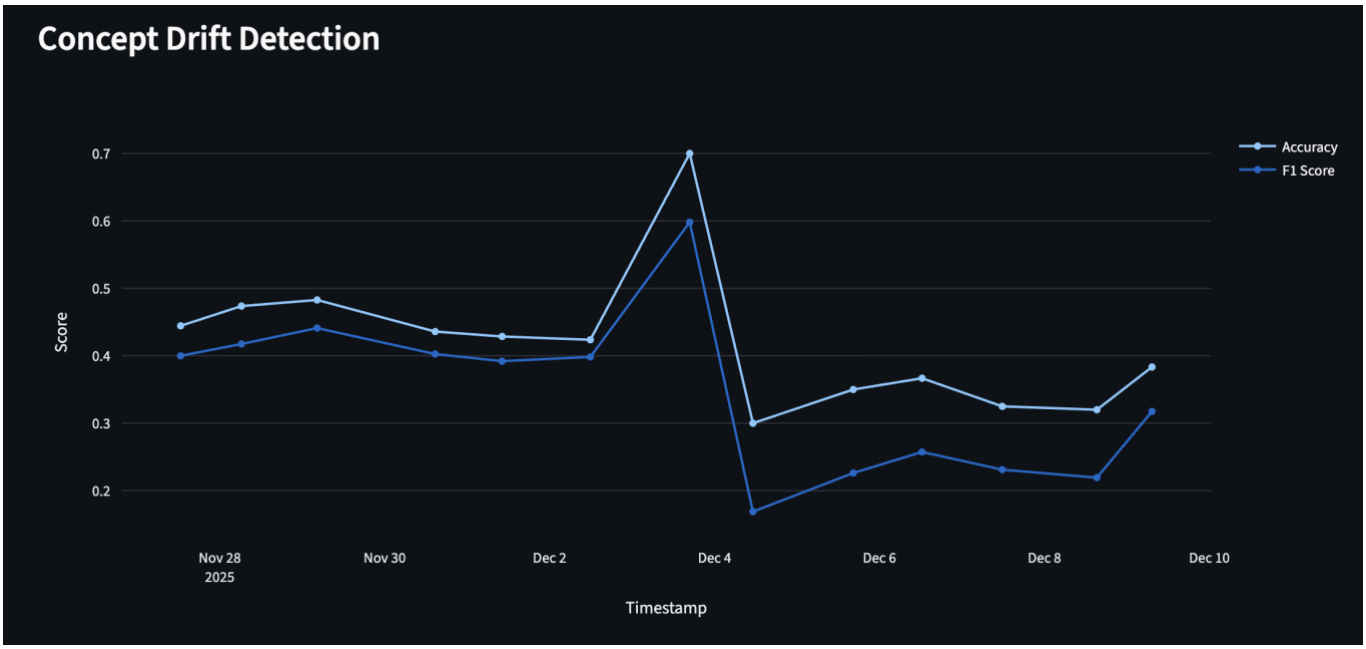


Figure 3636 Concept Drift Detection- Accuracy and F1 score trends monitored over the 2 weeks period

Concept Drift Analysis tracks accuracy and F1 score trends to detect changes in the feature-label relationship. The observed pattern—stable initial performance followed by high variance and degradation—suggests the model's learned decision boundaries from CREMA-D do not transfer reliably to RAVDESS.

This demonstrates the value of continuous monitoring: without ground-truth feedback from users, such performance degradation would go undetected in production systems. The findings recommend either domain adaptation techniques or retraining with multi-dataset augmentation to improve cross-corpus generalization.

5.14 Observability

We have used industry standard tooling to implement end-to-end observability for the Speech Emotion Recognition (SER) App:

- Logging (structlog + Promtail + Loki): The app emits structured JSON logs. Promtail tails these pod logs and pushes to Loki. Loki enables the logs to be queryable and linked to traces.
- Tracing (OpenTelemetry + Tempo): App is auto-instrumented (FastAPI/HTTPX/logging) and exports Opentelemetry protocol (OTLP) spans to Tempo, which ingests and stores the trace information.
- Metrics (Prometheus client + Prometheus): App exposes /metrics with counters/histograms and HTTP metrics middleware. We use Prometheus to scrape and stores these metrics
- Visualization & Correlation: Grafana is the observability platform which ingests the logging, tracing and metrics data from Loki, Tempo and Prometheus respectively. We have configured rich dashboards in Grafana to monitor the kubernetes infrastructure and application health for end-to-end observability, namely:
    - Kubernetes Dashboard
    - Speech Emotion Recognition (SER) App Dashboard

Figure 3737 Kubernetes Dashboard to monitor EKS cluster health

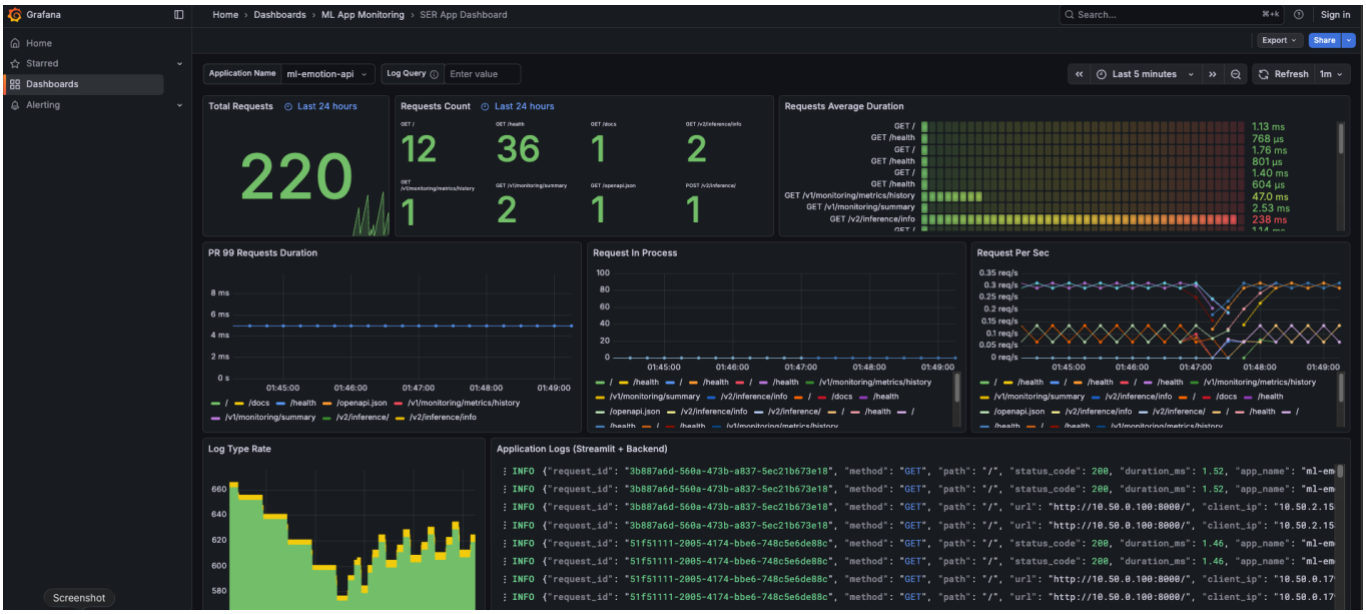


Figure 3838 Speech Emotion Recognition (SER) Application Dashboard

6. Future Work: Deep Learning Approaches

While our traditional ML approach achieved 82.22% accuracy, our analysis revealed challenges that deep learning is uniquely positioned to solve.

Our project demonstrated that success hinges on feature engineering and modelling complex interactions. Traditional ML requires us to hand-craft these features and interactions. Deep learning offers a more powerful approach by:

1. Learning features automatically from raw spectrograms, potentially discovering patterns our hand-crafted features missed.
2. Capturing long-range temporal dependencies more effectively than our fixed-window delta features.
3. Leveraging transfer learning from huge, pre-trained models to overcome speaker variability and data limitations.

7. References

Avci, U. (2025). A Comprehensive Analysis of Data Augmentation Methods for Speech Emotion Recognition. *IEEE Access. PP. 1-1. 10.1109/ACCESS.2025.3578143.*

Mehrabian, A. (1981). Silent Messages. *Belmont, Calif.: Wadsworth Pub. Co.*

Prajapati, Y. &. (2025). Systematic Evaluation of Deep Learning Paradigms for Speech Emotion Recognization Using Diverse Audio Sources. *1318-1330*.

Shoshika. (2025). Speech Emotion Recognition. *Kaggle - https://www.kaggle.com/code/shoshika/speech-emotion-recognition/notebook*.